

BidWise: Intelligent Reverse Bidding Platform

*A Project Report Submitted in
Partial Fulfilment of the Requirements
for the Degree of*

Bachelor of Technology

in

***Artificial Intelligence and Data Sciences
by***

M.V.SHABARISH (2303031241228)

A.SAI PRAKASH REDDY (2303031240230)

A.VISHNU VARDHAN (2303031240011)

K.VIJAY PRAKASH (2303031240549)

*Under the Guidance of
Dr. Sandeep Wadekar
Professor*



to

Artificial Intelligence and Data Science Department

***Parul University, Vadodara
April 2026***

UNDERTAKING

We declare that the work presented in this project report, entitled “BidWise: Intelligent Reverse Bidding Platform”, submitted to the Artificial Intelligence and Data Sciences Department, Parul University, Vadodara, for the award of the Bachelor of Technology degree in Artificial Intelligence and Data Sciences, is our original work. We have not plagiarised or submitted the same work for the award of any other degree. In case this undertaking is found incorrect, we accept that our degree may be unconditionally withdrawn.

April, 2026
Vadodara

M.V.SHABARISH (2303031241228): _____

A.SAI PRAKASH REDDY(2303031240230): _____

A.VISHNU VARDHAN (2303031240011): _____

K.VIJAY PRAKASH (2303031240549): _____



CERTIFICATE

This is to certify that the work contained in this project report entitled “**Improved Performance of Routing in Delay Tolerant Networks**”, by the following students bearing the respective Enroll numbers.

M.V.SHABARISH (2303031241228)

A.SAI PRAKASH REDDY(2303031240230)

A.VISHNU VARDHAN (2303031240011)

K.VIJAY PRAKASH (2303031240549)

This is also certified that this work is submitted for partial fulfilment of the degree of Bachelor of Technology in Artificial Intelligence and Data Sciences from Parul University, Vadodara, under my supervision from January 2026 to April 2026, and to the best of my knowledge, has not been submitted elsewhere for any degree.

Therefore, I deem this work fit and recommend it for submission for the award of the aforesaid degree.

April, 2026
Vadodara

Prof. SANDEEP WADEKAR,
Professor and Project Guide,
AIDS Department,
Parul University, Vadodara,

Synopsis

In recent years, the rapid growth of e-commerce and digital marketplaces has significantly transformed the way buyers and sellers interact. However, most existing platforms follow a fixed-price or limited negotiation model, which often leads to inefficient pricing, lack of transparency, and suboptimal decision-making for buyers. Additionally, buyers frequently struggle to determine whether they are receiving the best possible deal due to the absence of intelligent price comparison and recommendation systems.

BidWise addresses these challenges by introducing a reverse bidding platform where buyers can post their requirements, and multiple sellers compete by submitting bids. This approach enables competitive pricing and empowers buyers to select the most suitable offer based on factors such as price, seller rating, and delivery timelines. Unlike traditional systems that rely on static pricing, BidWise promotes a dynamic and interactive marketplace.

In conventional marketplace systems, decision-making is largely manual and lacks data-driven insights. Buyers must evaluate multiple options without structured comparison or intelligent recommendations. Furthermore, many systems do not incorporate real-time bidding mechanisms or provide transparency in the selection process.

In this project, we have designed and implemented a backend-driven system that supports real-time bidding, bid comparison, and user interaction management. The system ensures that each buyer can view bids specific to their projects, track live bidding durations, and make informed decisions. We also incorporated features such as role-based access (buyer/seller), secure authentication, and controlled review mechanisms.

Additionally, we propose an AI-based recommendation component that assists buyers in identifying an optimal price range based on historical data, market trends, and contextual factors such as demand and location. This enhances decision-making by providing intelligent insights rather than relying solely on manual comparison.

Unlike traditional approaches that depend on fixed pricing or uncontrolled message flooding (in bidding systems), our system maintains structured bid handling, ensuring efficiency, scalability, and clarity. The focus of this project is on improving decision quality, maximizing value for buyers, and creating a fair competitive environment for sellers.

Acknowledgements

We would like to express our gratitude to our supervisor, **Sandeep Wadekar**, who is a Professor in the Artificial Intelligence and Data Science Department at Parul University, Vadodara, for his advice, encouragement, enduring patience and constant support to me.

We are greatly indebted to all the faculty members of the Artificial Intelligence and Data Science Department of Parul University for their valuable input and for helping us understand theories and models along the bachelor's program.

We would like to express our sincere appreciation and gratitude to all our classmates for their encouragement, consistent support and invaluable suggestions at the time when needed. We remain grateful to all who directly or indirectly helped us at different stages of our project work.

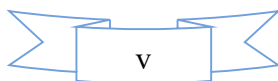
With deep appreciation, we recognise the invaluable contributions of all individuals and entities mentioned above, without whom the successful completion of this project would not have been possible.

M.V.SHABARISH (2303031241228)

A.SAI PRAKASH REDDY(2303031240230)

A.VISHNU VARDHAN (2303031240011)

K.VIJAY PRAKASH (2303031240549)



Contents

	Synopsis	iv
	Acknowledgement	v
1	Introduction	1
2	Routing in Delay Tolerant Network	8
2.1	DTN Network Model	8
2.2	Routing in Delay Tolerant Network	9
2.2.1	Routing Objective	9
2.2.2	Proactive Routing vs. Reactive Routing	10
2.2.3	Source Routing Vs Per-hop Routing	10
2.2.4	Message Splitting	10
2.2.5	Knowledge	11
2.2.5.1	Routing with zero knowledge	11
2.2.5.2	Routing with partial knowledge	12
2.2.6	Various Routing Techniques for DTN	13
3	Problem Statement	15
4	Background and Related Work	16
4.1	Mobile Ad Hoc Networking	16
4.2	Delay Tolerant Networking	17
5	Proposed Scheme	21
5.1	Algorithm Terminology	21
5.2	Algorithm Description	22

5.3	Algorithm	23
6	Simulation	26
6.1	Network Simulator (NS2)	26
6.2	Support of Wireless Communications	27
6.2.1	Mobile Node	28
6.3	Network Animator (Nam)	29
7	Simulation Results	30
7.1	Simulation Setup	30
7.2	Performance metrics	31
7.3	Simulation Parameters	32
7.4	Simulation Results	33
7.4.1	Effect of Node Density (Proposed Vs CAMR)	33
7.4.2	Effect of Traffic Load (Proposed Vs CAMR)	37
7.4.3	Effect of Storage Capacity and Traffic Load on Delivery Ratio	40
8	Conclusion and Future Work	43
	References	45

List of Figures

1	Example DTN network	2
2	DakNet Concept	4
3	The Internet Motoman Project in Cambodia	5
4	Edges in a DTN graph	8
5	Conceptual Performance Vs Knowledge Trade-off	11
6	Mobile Node Structure	29
7	Effect of Node Density on Delivery Ratio	34
8	Effect of Node Density on Average Delay	35
9	Effect of Node Density on Efficiency	36
10	Effect of Traffic Load on Delivery Ratio	38
11	Effect of Traffic Load on Average Delay	39
12	Effect of Traffic Load on Efficiency	40
13	Effect of Storage Capacity on Delivery Ratio on varying Traffic Load ...	41
14	Effect of Traffic Load on Delivery Ratio on varying Storage Capacity ...	42

List of Tables

1	Overview of different routing algorithms	13
2	Simulation Parameters	32
3	Effect of Node Density on Delivery Ratio	34
4	Effect of Node Density on Average Delay	35
5	Effect of Node Density on Efficiency	36
6	Effect of Traffic Load on Delivery Ratio	37
7	Effect of Traffic Load on Average Delay	38
8	Effect of Traffic Load on Efficiency	39
9	Effect of Storage Capacity on Delivery Ratio on varying Traffic Load ...	41
10	Effect of Traffic Load on Delivery Ratio on varying Storage Capacity ...	42

Chapter 1

Introduction

Traditional procurement and service outsourcing workflows often rely on informal negotiations, static quotations, and limited transparency in vendor selection. As digital marketplaces have grown, buyers now expect faster sourcing cycles, price visibility, and measurable quality signals. A reverse bidding system addresses this gap by allowing buyers to post a project requirement while sellers compete by offering lower prices or better delivery commitments. This shifts the market from seller centric pricing to buyer centric decision making while ensuring competitive fairness across vendors.

BidWise is a full stack reverse bidding marketplace designed to operationalize this model as a complete, real time platform. Unlike generic freelance portals, BidWise enforces a structured project lifecycle: buyers create projects with clear scope, budget, and deadline; sellers submit bids with delivery time and proposals; buyers compare bids in a project specific table; and a single accepted bid triggers contract and order generation. This ensures that the bidding process is not isolated but connected to project execution, payment initiation, and delivery confirmation.

A reverse auction marketplace introduces several challenges. First, requirements must be sufficiently clear so sellers can bid accurately and fairly. Second, buyers need objective comparison tools to evaluate bids beyond price alone. Third, once a bid is accepted, the system must guarantee transactional integrity ensuring that only one bid is accepted, all others are rejected, and the project status transitions to execution. Fourth, payment and delivery workflows must be auditable, with notifications and status updates available to both parties. BidWise addresses these challenges through structured project creation, multi criteria bid comparison, real time dashboards, and a complete order lifecycle integrated with payment processing and notifications.

From a software systems perspective, BidWise combines a modern frontend (Next.js, Tailwind, ShadCN UI) with a secure backend (Spring Boot, JPA/Hibernate, JWT security, MySQL). The platform integrates Stripe for payment intents and supports in app plus email notifications. This architecture enables role based dashboards for buyers, sellers, and administrators while maintaining data consistency across bids, projects, orders, and payments.

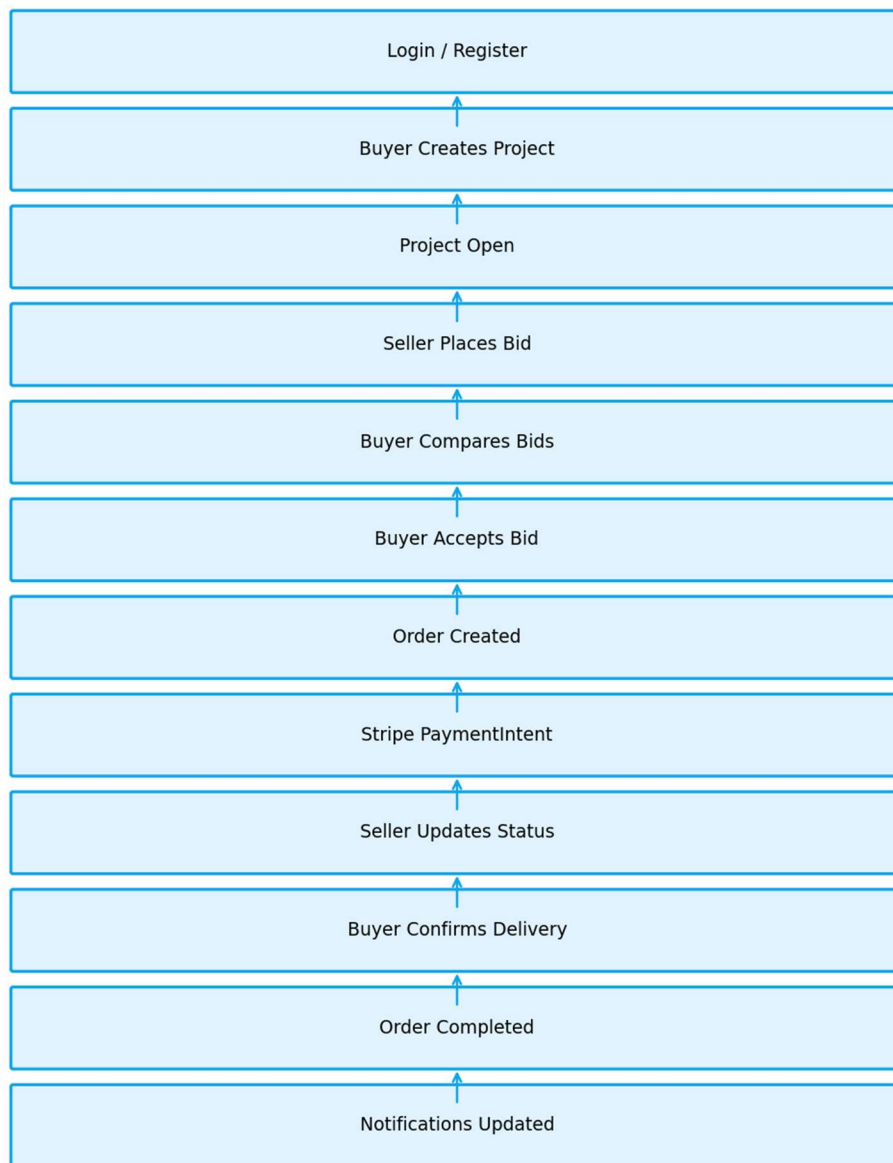


Figure 1: BidWise Reverse Bidding Workflow

The rise of platform based procurement reflects a broader trend in digital transformation where organizations seek faster vendor discovery, auditable decision trails, and lower transaction

overhead. Traditional tender processes often rely on email exchanges and manual comparison of quotations, which increases the risk of errors, delayed decisions, and biased selection. BidWise replaces these ad hoc processes with a structured, data driven workflow, ensuring that every bid, status change, and acceptance decision is recorded in the database and available for audit.

In the Indian context, small and medium enterprises (SMEs) increasingly depend on digital sourcing to access wider supplier networks and secure competitive pricing in INRbased transactions. A reverse bidding model expands supplier access for buyers while giving sellers a clear, transparent opportunity to compete on both price and delivery commitments. BidWise addresses this by combining bidding with performance based trust signals such as ratings, reviews, and completion history, thereby supporting sustainable buyer seller relationships.

Another critical requirement in real world procurement is post ward governance. BidWise integrates order tracking, delivery confirmation, and payment initiation so that procurement does not stop at bid selection. This end to end pipeline reduces ambiguity after award, enables timely issue resolution, and strengthens accountability for both parties. The inclusion of admin moderation and dispute management further ensures platform reliability and compliance readiness as usage scales.

The remainder of this report is organized as follows.

Chapter 2 discusses reverse bidding systems and related research.

Chapter 3 defines the problem statement addressed by BidWise.

Chapter 4 presents background and existing marketplace practices.

Chapter 5 outlines the proposed system design and architecture.

Chapter 6 details the implementation across frontend, backend, and integrations.

Chapter 7 summarizes testing and validation.

Chapter 8 concludes with outcomes and future enhancements.

Chapter 2: Literature Survey

2.1 Related Research Works and Systems

2.1.1 Study 1: Evolution of Electronic Reverse Auctions (Bentaleb, 2020)

Bentaleb (2020) presents an extensive review of how electronic reverse auctions (eRAs) have evolved from price only bidding instruments into decision systems that incorporate quality, delivery, and risk factors. The study frames reverse auctions within the broader transformation of procurement from transactional purchasing to strategic sourcing. In earlier implementations, reverse auctions were primarily cost reduction mechanisms, and the buyers decision logic was dominated by the lowest price. Bentaleb argues that such a singular focus often introduces hidden costs due to supplier performance issues, delayed delivery, and quality non conformance.[1]

The paper highlights that modern procurement increasingly values total cost of ownership rather than unit price alone. Factors such as delivery reliability, post sale service, compliance with standards, and supplier stability influence the real value of a procurement decision. Bentaleb suggests that reverse auction platforms should therefore support multi attribute bidding, where suppliers submit structured offers that cover multiple performance dimensions. This is a critical shift because it converts auctions from purely competitive price wars into structured multi criteria evaluations that align with organizational priorities. [1]

A key discussion point is the design of scoring models. The study emphasizes that scoring must balance transparency with decision flexibility. If scoring rules are too rigid, they may exclude potentially valuable suppliers who do not perfectly match predefined criteria. Conversely, if scoring rules are vague, suppliers may perceive the process as unfair. Bentaleb recommends clearly published scoring criteria with adjustable weights so that buyers can reflect strategic priorities while maintaining bidder trust. This approach reduces post award disputes and strengthens supplier confidence in the auction process. [1]

The study also explores market dynamics and supplier behavior. Suppliers adjust bidding strategies when they know that price is only one component of evaluation. Instead of racing toward unsustainable prices, they may emphasize reliability, faster delivery, or higher service guarantees. This can lead to healthier marketplaces where quality suppliers remain engaged over long periods. Bentaleb notes that when the evaluation framework is aligned with long term value, reverse auctions can become more sustainable and less adversarial. [1]

On the system side, Bentaleb stresses the importance of integrating auctions with contract and order workflows. Reverse auctions are not isolated events; they initiate procurement cycles that require contract generation, order execution, and performance monitoring. The study advocates for architectures that retain full bid histories and decision rationales to enable audit trails and compliance checks. This aligns with enterprise procurement best practices, particularly in regulated industries where decision transparency is mandatory. [1]

Another focus is data quality and decision support. Multi criteria bidding requires accurate supplier metadata, reliable ratings, and consistent measurement of delivery and performance outcomes. Without this data, multi criteria scoring becomes arbitrary. Bentaleb recommends systems that continuously capture supplier performance metrics and incorporate them into future evaluations. Such feedback loops allow procurement systems to improve decision accuracy over time. [1]

Bentaleb also recognizes challenges. Multi criteria auctions can be more complex for suppliers, especially if the scoring method is not intuitive. There is a risk that increased complexity reduces participation or encourages gaming. To mitigate this, the study suggests that platforms should provide clear guidance, standard bid forms, and transparency in evaluation logic. Simplicity in the interface combined with robust internal scoring is the recommended balance. [1]

From a practical perspective, the study's insights align closely with the design of BidWise. BidWise does not rely solely on price; it presents delivery time and seller ratings alongside bid price and computes a recommended score. This mirrors the multi criteria evaluation framework proposed in the study. BidWise also keeps bid histories, order status updates, and reviews, creating the performance feedback loop that Bentaleb describes. [1]

The study ultimately concludes that reverse auctions can remain powerful sourcing tools when they are designed as value oriented decision systems rather than price only competitions. BidWise operationalizes this conclusion by combining auction logic with structured evaluation metrics, thereby embedding modern procurement best practices into a real time marketplace platform. [1]

2.1.2 Study 2: Reverse Auction Appropriateness in B2B Sourcing (Hawkins et al., 2009)

Hawkins et al. (2009) provide an empirical investigation into when reverse auctions are most appropriate for business to business sourcing. The study emphasizes that reverse auctions are not universally beneficial and must be applied under the right sourcing conditions. The authors identify requirement clarity, supplier competition, switching costs, and relationship complexity as the most significant determinants of reverse auction suitability. Their analysis highlights that reverse auctions perform best in standardized procurements where multiple suppliers can compete on comparable terms. [2]

Requirement specificity emerges as the strongest predictor. If a buyer can define the scope, quality standards, delivery timelines, and acceptance criteria clearly, suppliers can bid competitively without excessive risk. This leads to fair and realistic pricing and reduces the probability of post award disputes. Conversely, ambiguous requirements can lead to under bidding followed by renegotiations or execution failure. Hawkins et al. argue that procurement systems should enforce structured requirement fields to maintain auction integrity. [2]

Supplier market structure is another key variable. Auctions thrive when there is sufficient supplier competition, as multiple vendors exert price pressure on each other. If the supplier base is too small or highly specialized, the auction may result in minimal price movement and could even harm buyer supplier relationships. The study advises procurement systems to assess supplier availability before deciding to conduct a reverse auction. This supports the need for platforms like BidWise to grow and maintain a healthy seller ecosystem. [2]

Switching cost is also examined. When switching costs are low, buyers can readily select a new supplier without major operational disruption. In such cases, reverse auctions can deliver strong

cost savings. When switching costs are high, however, price competition is constrained and reverse auctions may create friction without significant benefit. Hawkins et al. recommend that reverse auctions be used primarily in low switching cost categories or where suppliers offer interchangeable services. [2]

Relationship complexity further influences auction appropriateness. For strategic partnerships where collaboration, co-innovation, or long term alignment is important, reverse auctions may undermine trust. Hawkins et al. suggest that negotiation based sourcing is more suitable in such contexts. Reverse auctions are best positioned for transactional sourcing, where efficiency is prioritized over relationship depth. This is why BidWise focuses on project based service procurement with clearly measurable outcomes rather than deep strategic alliances. [2]

Methodologically, the study applies empirical modeling to validate these determinants. It uses survey and regression analysis to correlate auction success with sourcing attributes. The findings consistently indicate that requirement clarity and supplier competition are decisive factors. This provides strong justification for BidWise structured project creation workflow, which includes explicit fields for scope, deadlines, budgets, categories, and attachments. [2]

The study also emphasizes governance and risk mitigation. Auctions should be integrated with supplier qualification, evaluation criteria, and post award monitoring. Hawkins et al. observe that auctions without governance can lead to supplier dissatisfaction and performance issues. BidWise addresses this by incorporating review systems, order tracking, and delivery confirmation processes. These mechanisms ensure that auction outcomes are validated by execution performance. [2]

Another observation is that transparency in the auction process builds supplier trust. When suppliers understand the rules and can verify that evaluation is fair, participation increases. BidWise supports this by displaying bid history, status changes, and clear acceptance/rejection outcomes. Such transparency reduces disputes and supports long-term marketplace health. [2]

Overall, Hawkins et al.'s work demonstrates that reverse auctions are effective when supported by clear requirements, competitive supplier markets, and structured governance. BidWise aligns with these criteria by enforcing requirement structure at project creation, enabling broad seller

participation, and providing post award tracking and feedback loops. The platform therefore reflects the empirical conditions under which reverse auctions succeed. [2]

2.1.3 Study 3: Procurement Auctions vs. Negotiations (Wu & Kersten, 2017)

Wu and Kersten (2017) analyze procurement auctions in comparison with negotiation based sourcing. The study emphasizes that auctions are effective at reducing cycle time and price, but negotiations may support stronger supplier relationships and flexibility. The authors position auctions as efficiency oriented mechanisms, particularly suitable for standard procurements where the buyer's priorities are known and supplier capabilities are relatively comparable. [3]

One of the central findings is that auctions significantly compress procurement timelines. By structuring bids in a competitive environment, buyers can quickly evaluate offers without extended negotiation rounds. This makes auctions appealing when speed is important or when many suppliers can bid simultaneously. BidWise reflects this strength through project wise bid comparison tools and automated bid acceptance workflows, enabling buyers to reach decisions quickly. [3]

The study also discusses supplier behavior in auction settings. Suppliers may under bid to win contracts, which can later result in scope disputes or quality compromises. This risk indicates the importance of balancing price with supplier performance indicators. Wu and Kersten argue that multi criteria evaluation improves procurement outcomes by rewarding reliability and quality alongside cost. BidWises rating and delivery time evaluation aligns with this recommendation. [3]

Negotiation based sourcing is shown to be valuable when requirements are complex or when collaboration is central to success. Negotiations allow for iterative refinement of requirements and can build trust between buyer and supplier. The authors suggest that auction platforms

should integrate communication features to mitigate the rigidity of auctions. BidWise addresses this by providing direct buyer seller messaging, enabling clarification and alignment during the bidding process and after award. [3]

Another aspect explored is the long term relationship impact. Auctions can sometimes reduce perceived supplier fairness if they are overly price driven, potentially discouraging participation in future bids. Wu and Kersten propose that transparency and post award evaluation help maintain supplier trust. BidWise's transparent bid status updates and review system provide this feedback mechanism, helping sellers understand decision outcomes and improve future bids. [3]

The study further emphasizes the importance of integration with payment and contract processes. Auctions should feed seamlessly into contractual execution to avoid operational gaps. Wu and Kersten note that procurement efficiency is maximized when systems manage the entire lifecycle, from bid to payment. BidWise implements this through automatic order creation, payment intent integration (Stripe), and delivery confirmation workflows, thereby operationalizing the full cycle. [3]

From a governance perspective, the authors highlight auditability as a key advantage of digital auctions. Bid histories, acceptance decisions, and evaluation criteria provide an auditable trail that supports compliance and accountability. BidWise maintains bid history, order status logs, and notifications, enabling both operational transparency and evidence for disputes or audits. [3]

In summary, Wu and Kersten's findings support a balanced approach: use auctions for efficiency while preserving collaboration and governance through communication and lifecycle management. BidWise embodies this approach by combining auction mechanics with messaging, post award order tracking, and review based accountability. The platform thus reflects the hybrid best practice recommended in the study. [3]

2.1.4 Related Systems and Industry Practices

Freelance and procurement platforms such as Upwork, Fiverr, and Freelancer provide digital marketplaces that support project posting, bidding, contracts, and payments. These platforms demonstrate the importance of trust mechanisms, seller reputation, and secure payment systems. However, many of these systems are optimized for open marketplace discovery rather than

reverse auction driven procurement. BidWise differs by enforcing project specific reverse bidding workflows and structured comparison for buyer decision making.

Enterprise procurement tools incorporate supplier qualification, compliance checks, contract lifecycle management, and performance tracking. BidWise adopts several of these best practices by enforcing role based access, order tracking, and review history. While it does not implement full enterprise contract management, its design provides a scalable foundation for those capabilities.

2.2 Comparison with the Implemented System

BidWise operationalizes research backed auction practices by supporting structured project scopes, competitive supplier participation, and objective bid comparison. Unlike systems that only show bid price, BidWise incorporates multiple decision dimensions (price, delivery, rating, historical completion), which directly reflects the multi criteria evaluation suggested in the literature. The platform further bridges auction and negotiation models through messaging and a post award order lifecycle, enabling collaboration without sacrificing auction transparency.

2.3 Summary Comparison Table

Source	Method / Scope	Key Findings	Observed Gap	BidWise Alignment
Bentaleb (2020)	Multi criteria reverse auctions	Price only auctions reduce quality; multi criteria scoring improves outcomes	Most platforms still price dominant	BidWise uses rating, delivery, and AI score
Hawkins et al. (2009)	Empirical B2B sourcing study	Auctions are suitable for specifiable requirements and competitive markets	Weak support for requirement clarity	BidWise enforces structured project requirements
Wu & Kersten (2017)	Auction vs negotiation	Auctions lower cost and time; negotiations	Few systems support hybrid flow	BidWise adds messaging and order

		improve relationships		collaboration
Upwork/Freelancer	Freelance marketplaces	Strong ratings and payments but not reverse auction focused	No project wise reverse auction logic	BidWise is reverse bidding first

Chapter 3

Analysis / Software Requirements Specification (SRS)

3.1 Purpose of the SRS

This chapter provides a comprehensive Software Requirements Specification for the BidWise reverse bidding marketplace. The SRS documents the functional and non functional requirements, user classes, constraints, interfaces, data structures, and acceptance criteria for the implemented system. It serves as a formal baseline for validating that the completed product meets academic and practical expectations, and that each workflow from project creation to payment and order completion is supported by a traceable requirement.

The SRS is written to be self contained and implementation aligned. It reflects the actual modules built in BidWise: buyer and seller dashboards, bid comparison, bid acceptance workflow, order lifecycle, Stripe payment integration, notifications, reviews, and administration controls. The content is structured to enable verification, testing, and future maintenance without ambiguity.

3.2 Product Scope

BidWise is a reverse bidding marketplace platform that allows buyers to create projects and receive competitive bids from multiple sellers. It supports project wise bid comparison, a single acceptance rule for bids, order creation, payment initiation, delivery confirmation, and review/feedback capture. The platform enforces role based access, audit trails, and consistent state transitions for bids, projects, orders, and payments.

The system is designed for service based procurement scenarios such as web development, design, marketing, analytics, and software maintenance, where requirements can be structured and evaluated against multiple criteria. The scope includes buyer, seller, and admin interfaces, plus integrations for payments (Stripe) and email notifications (Spring Mail). It explicitly excludes advanced enterprise contract negotiation and legal compliance modules, which are documented as future enhancements.

3.3 Stakeholders and User Classes

The primary stakeholders are Buyers, Sellers, and Platform Administrators. Secondary stakeholders include project supervisors, institutional evaluators, and system maintainers who require auditability of actions. Buyer users focus on project creation, bid evaluation, and order tracking. Seller users focus on project discovery, bid submission, and order execution. Administrators focus on governance: user verification, project moderation, and dispute resolution.

User classes are segmented by role with strict access control. Buyers cannot access seller only bidding actions; sellers cannot accept bids or modify buyer owned projects; administrators can view and moderate all resources but do not interfere in bid acceptance decisions. These separations enforce both security and operational consistency.

Table 3.1 Role Access Matrix

Module	Buyer	Seller	Admin
Create Project	Yes	No	View
View Open Projects	Yes (own)	Yes	Yes
Place Bid	No	Yes	No
Compare Bids	Yes (own)	No	View
Accept/Reject Bid	Yes (own)	No	No
Order Status Update	Confirm Delivery	Update	View
Payments	Initiate	View	View
Messaging	Yes	Yes	View
Moderation & Disputes	No	No	Yes

3.4 Operating Environment

The system operates as a web based SaaS application. The frontend runs on modern browsers (Chrome, Edge, Firefox) and is responsive for desktop and tablet use. The backend is a REST API implemented in Spring Boot, running on JDK 17+ with a MySQL 8+ database. Stripe and SMTP services are accessed over HTTPS with secure credentials stored as environment variables.

The platform is optimized for mid scale concurrent usage (tens to hundreds of users) in an academic and SME procurement context. The architecture separates concerns to enable scale out by splitting web and API tiers and adding caching or message queues in future iterations.

3.5 Assumptions and Dependencies

Key assumptions include:

- (1) Buyers provide reasonably structured project requirements;
- (2) Sellers submit bids in good faith;
- (3) Stripe and email services are reachable during transactions;
- (4) All users access the platform through authenticated sessions. Dependencies include Stripe API for payments, SMTP for email, and database availability for consistency.

3.6 Functional Requirements

Functional requirements define the system behavior across modules. Each requirement is implemented and verified in the current system.

Table 3.2 Functional Requirements

ID	Requirement
FR 01	Buyer can register/login with role based access
FR 02	Buyer can create a project with title, description, budget, category, deadline, and attachments
FR 03	Seller can browse open projects with filters
FR 04	Seller can place a bid with price, delivery time, and proposal
FR 05	Buyer can view project specific bid list and comparison table
FR 06	Buyer can accept one bid and reject all others
FR 07	System creates an Order and updates Project status to IN_PROGRESS on acceptance
FR 08	Seller updates order status (PREPARING

	SHIPPED ? DELIVERED)
FR 09	Buyer confirms delivery and completes order and project
FR 10	Payment intent created through Stripe and tracked in Payment table
FR 11	In app notifications stored and retrievable by user
FR 12	Email notifications sent on bid acceptance, payment confirmation, and order shipped
FR 13	Buyer and seller can exchange messages
FR 14	Buyer and seller can leave reviews after completion or rejection
FR 15	Admin can view users, verify sellers, block users, and moderate projects
FR 16	Admin can resolve disputes and view system metrics

3.7 Non Functional Requirements

Non functional requirements define quality attributes of BidWise, ensuring security, reliability, and maintainability.

Table 3.3 Non Functional Requirements

Category	Requirement
Security	JWT authentication, role based authorization, and secure password hashing
Performance	Dashboard and listing pages load within 2?3 seconds under normal load
Availability	System uptime targeted at >99% in production deployment
Scalability	Stateless API design allows horizontal scaling
Reliability	Transactional integrity for bid acceptance and order creation
Maintainability	Layered architecture (Controller Service Repository)
Usability	Consistent SaaS UI, responsive layout, clear navigation
Auditability	Bid history and order status changes stored for traceability

3.8 Business Rules

BR-01: A project can have many bids, but only one bid can be accepted.
BR-02: When a bid is accepted, all other bids for that project must be rejected.
BR-03: Orders are created only after a bid is accepted and must reference the accepted bid.
BR-04: Buyers can confirm delivery only after order status is DELIVERED.
BR-05: Payments are initiated only for accepted bids and linked orders.
BR-06: Admins can moderate but cannot alter bid acceptance outcomes.

3.9 Data Requirements and Data Dictionary

BidWise stores structured entities to maintain workflow state. The following table summarizes primary entities and their purpose.

Table 3.4 Data Dictionary

Entity	Key Fields	Purpose
User	id, name, email, role, rating	Stores buyer/seller/admin accounts
Project	id, buyerId, title, budget, deadline, status	Defines project scope and lifecycle
Bid	id, projectId, sellerId, price, status	Captures seller offers
Order	id, bidId, buyerId, sellerId, status	Tracks execution after acceptance
Payment	id, orderId, amount, status, stripeIntentId	Tracks payment lifecycle
Notification	id, userId, title, isRead	Stores alerts for users
Review	id, fromUserId, toUserId, rating, comment	Captures feedback
Dispute	id, projectId, raisedBy, status	Admin dispute handling

3.10 External Interface Requirements

External interfaces include REST APIs, Stripe payments, and SMTP email. The system exposes secure REST endpoints for authentication, project management, bidding, orders, payments, notifications, and admin actions.

Table 3.5 External APIs and Integrations

Interface	Purpose
Stripe PaymentIntent	Initiate and confirm payments
Stripe Webhook	Update payment status
SMTP (Spring Mail)	Send transactional emails
REST API	Frontend backend communication

3.11 User Interface Requirements

UI requirements focus on clarity, responsiveness, and role based access. Dashboards must show active projects, bids, and orders with clear status indicators. Bid comparison should be project specific and sortable. Order details should show payment status and delivery progress. Admin screens must provide user and project moderation controls.

Figure 3.1: Buyer Dashboard Layout

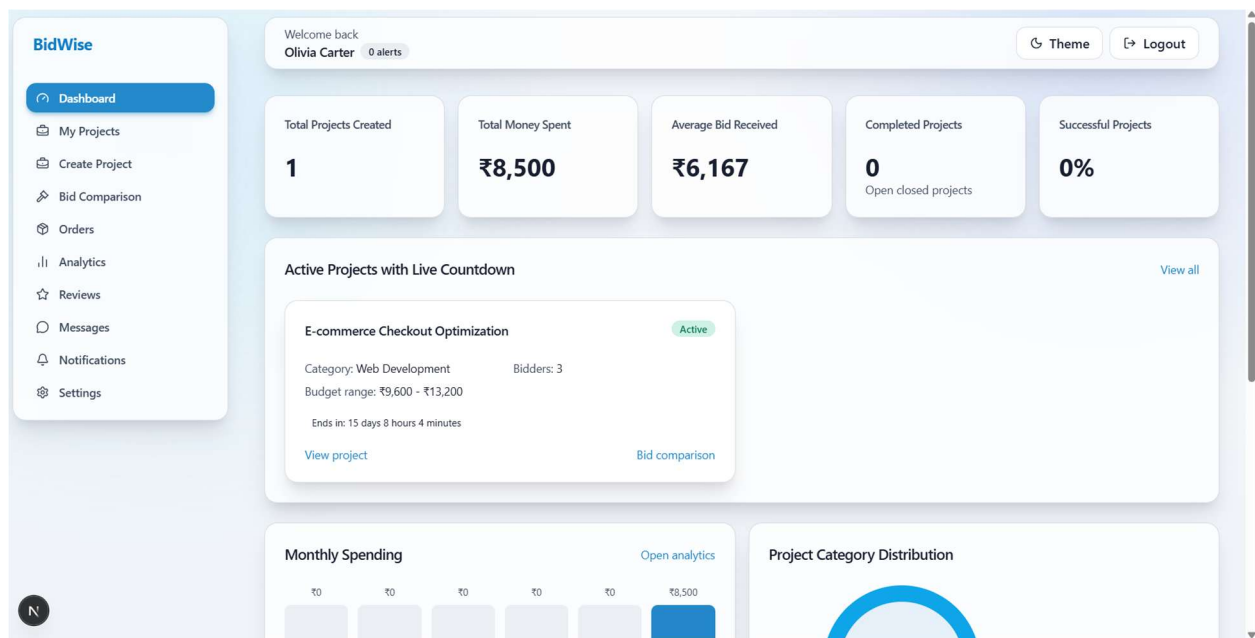


Figure 3.2: Bid Comparison Table

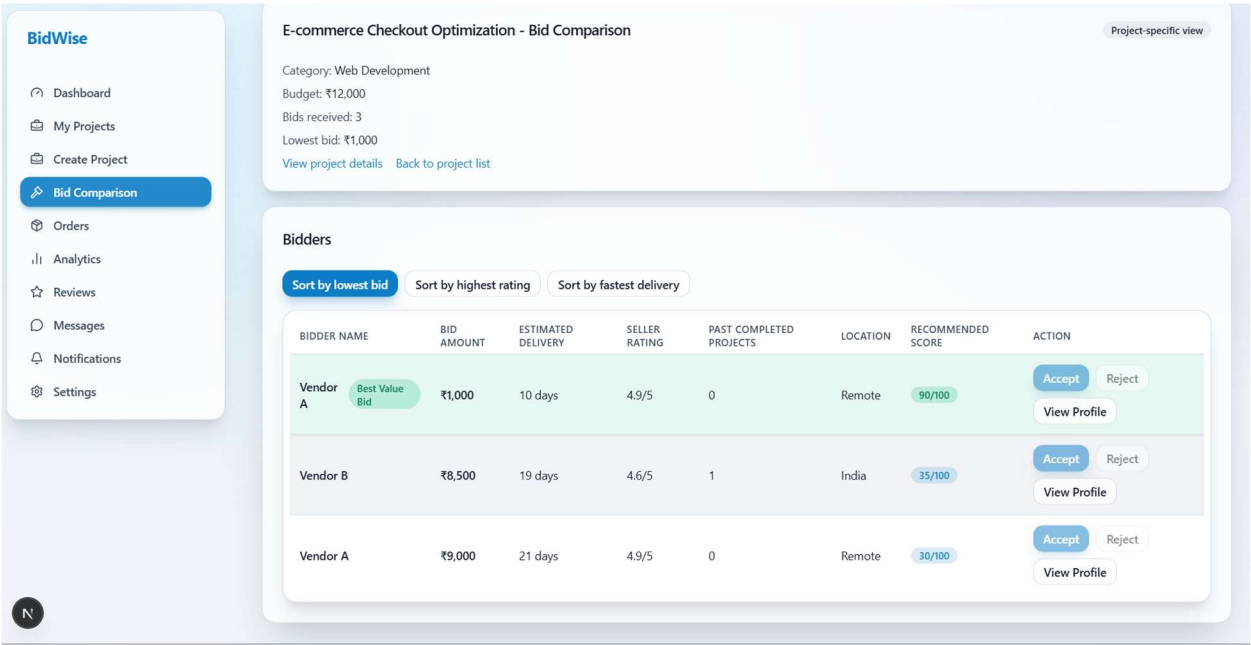
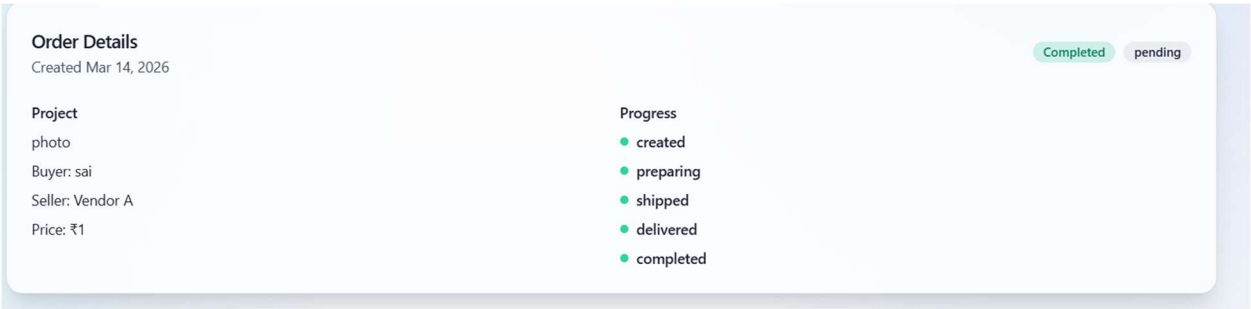


Figure 3.3: Order Status Timeline (Placeholder)



3.12 Hardware and Software Requirements

Minimum hardware requirements: 8 GB RAM, dual core CPU, 10 GB storage. Recommended: 16 GB RAM for development. Software requirements include Node.js 18+, JDK 17+, MySQL 8+, and a modern browser.

3.13 Requirements Traceability Matrix

Table 3.6 Traceability Matrix

Objective	Mapped Requirements
Enable reverse bidding	FR 02, FR 04, FR 05
Ensure single accepted bid	FR 06, FR 07
Provide order lifecycle	FR 07, FR 08, FR 09
Integrate payments	FR 10
Enable notifications	FR 11, FR 12
Admin governance	FR 15, FR 16

3.14 Acceptance Criteria

The system is accepted when: (1) Buyers can create projects and receive bids; (2) Sellers can place bids and view statuses; (3) Bid acceptance updates all states consistently; (4) Orders progress through status updates; (5) Payments are created and tracked; (6) Notifications and email alerts are delivered; (7) Admin actions apply without data loss.

3.15 References Used in Analysis

References include procurement auction research papers, platform case studies, and technology documentation for Spring Boot, Next.js, Stripe, and MySQL.

Chapter 4

System Design

4.1 Design Overview

The BidWise system is designed as a multi tier SaaS application that separates presentation, business logic, and persistence. The design prioritizes consistency of bid acceptance, auditability of decision flow, and usability for multiple roles. The architecture follows a client server model with a stateless REST API layer, enabling scalability and clear separation of responsibilities.

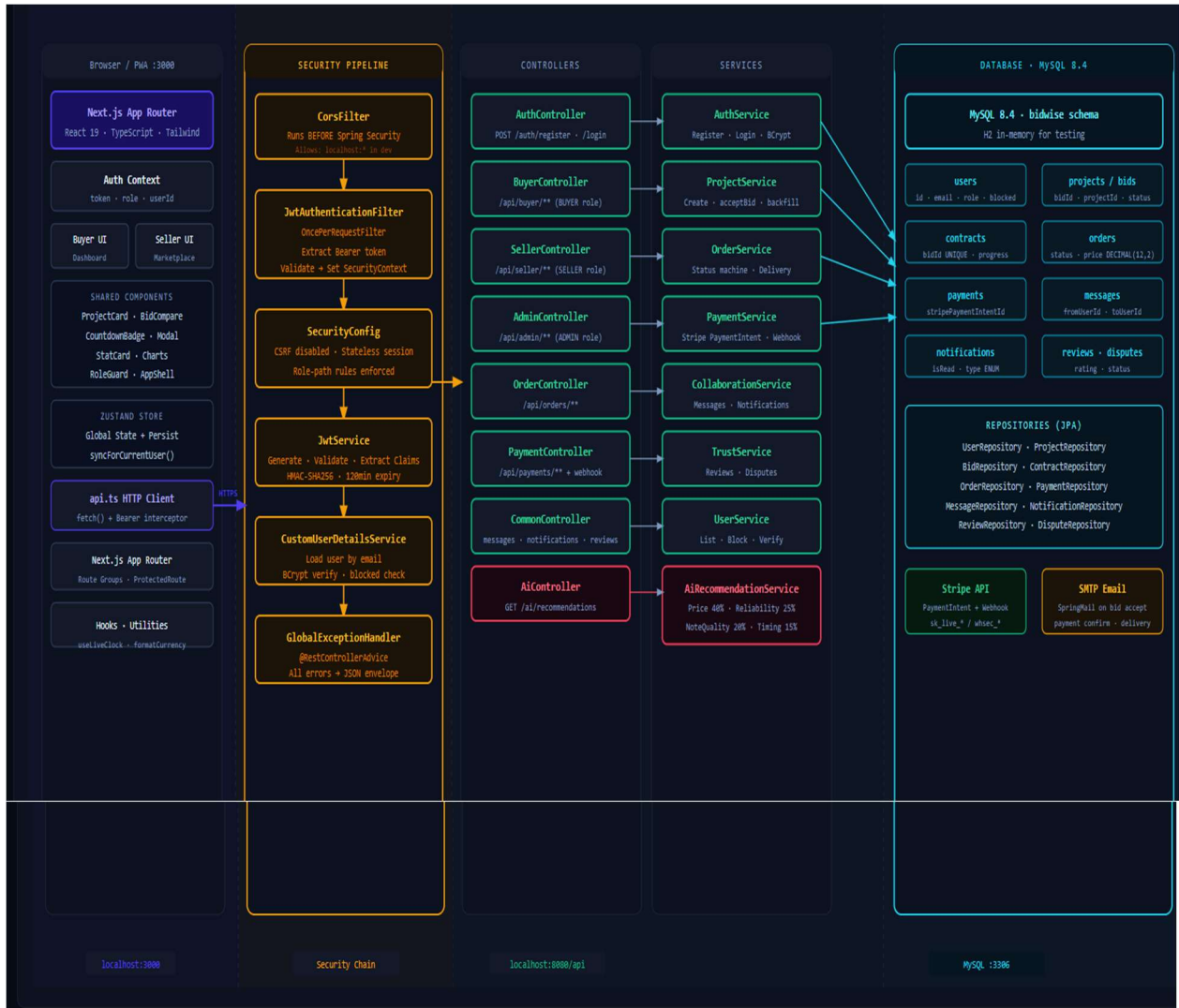
Key design principles include:

- (1) role based isolation of features, ensuring that buyers, sellers, and administrators see only their permitted actions;
- (2) transactional integrity for bid acceptance and order creation;
- (3) clear lifecycle management for projects, bids, orders, and payments; and
- (4) integration readiness for external services such as Stripe and SMTP. The design also emphasizes traceability, allowing every state change to be logged and verifiable.

4.2 Overall System Architecture

BidWise uses a layered architecture: a Next.js frontend acts as the client, a Spring Boot backend exposes REST APIs, and a MySQL database stores transactional data. Stripe and SMTP services are integrated through service adapters. The frontend communicates with the backend using authenticated requests (JWT tokens). All business rules are enforced serverside to ensure security and data integrity.

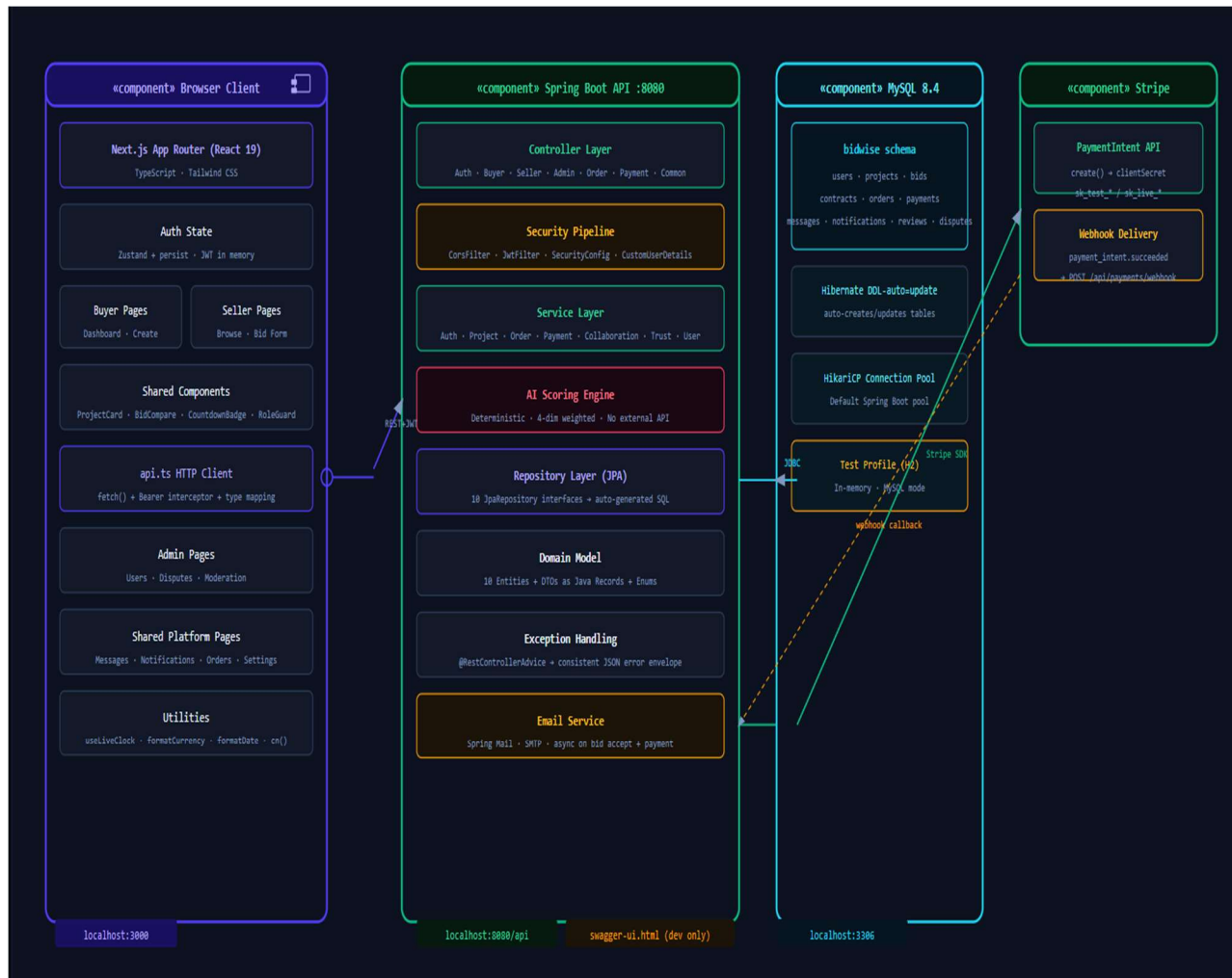
Figure 4.1 shows the overall system architecture including integrations.



4.3 Component Decomposition

The platform is decomposed into major components: Authentication & User Management, Project Management, Bidding Engine, Order Management, Payment Management, Notification System, Review System, and Admin Governance. Each component maps to dedicated service classes and REST endpoints. This modular decomposition enables independent testing and future extensibility.

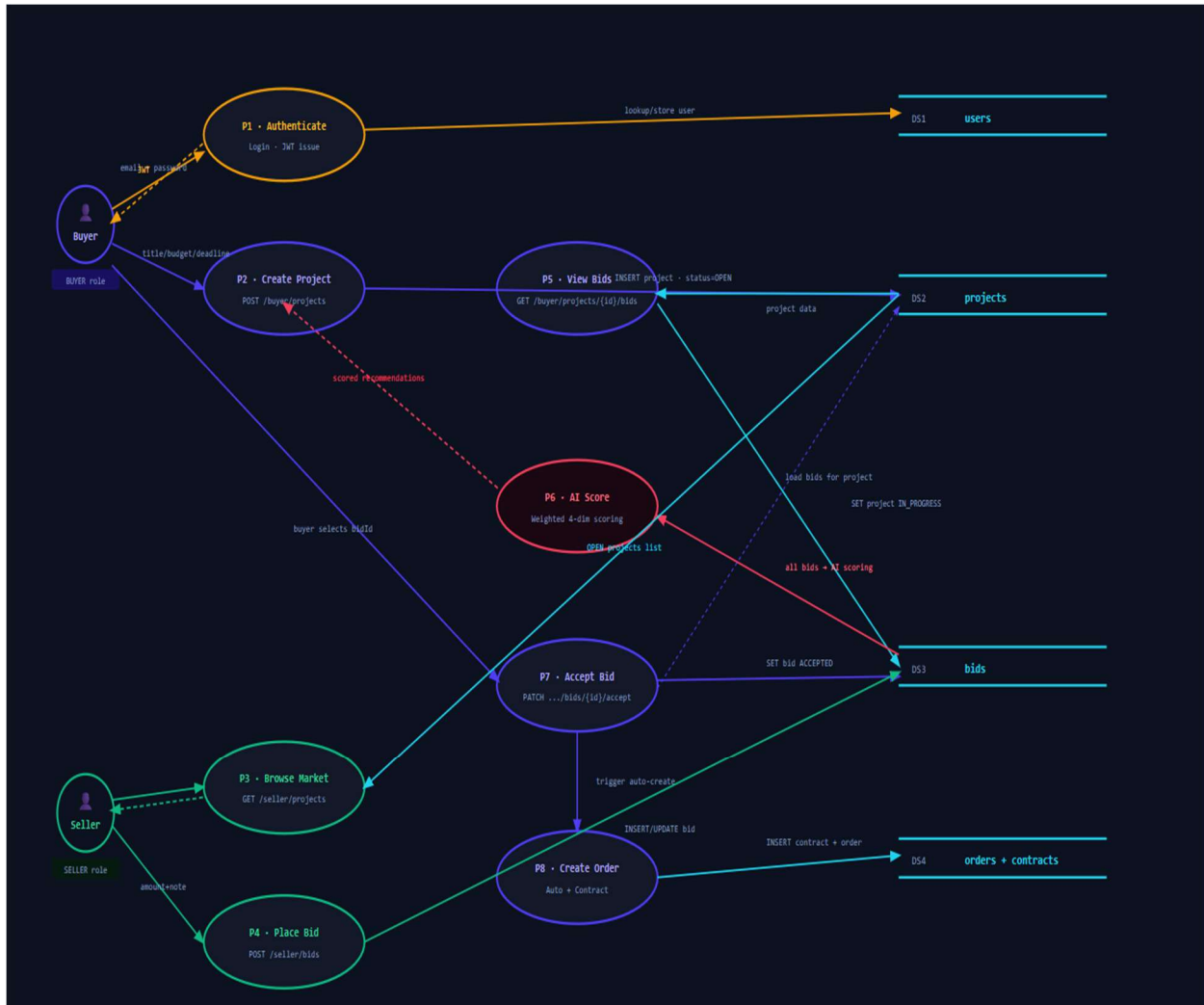
Figure 4.2: Component Diagram / Module Decomposition



4.4 Data Flow Design

Data flows across the system follow a request response pattern. Buyers create projects which are persisted in the Project table. Sellers view open projects and submit bids stored in the Bid table. Buyers accept bids, triggering transactional updates: bid statuses update, project status changes, an Order is created, and notifications are emitted. Stripe integration creates payment intents, stored in the Payment table, and webhook callbacks update payment status. These flows are captured in sequence and data flow diagrams to demonstrate lifecycle consistency.

Figure 4.3: Data Flow Diagram



4.5 Data Model / ER Design

The data model is centered around Project, Bid, Order, and User entities. Project is owned by a buyer, Bids reference both project and seller, and Orders are created only for accepted bids. Payment records are linked to orders, while Notifications capture system events for each user. Reviews and disputes provide post award governance. This relational design ensures normalized storage and consistent referential integrity.

Figure 4.5: ER Diagram



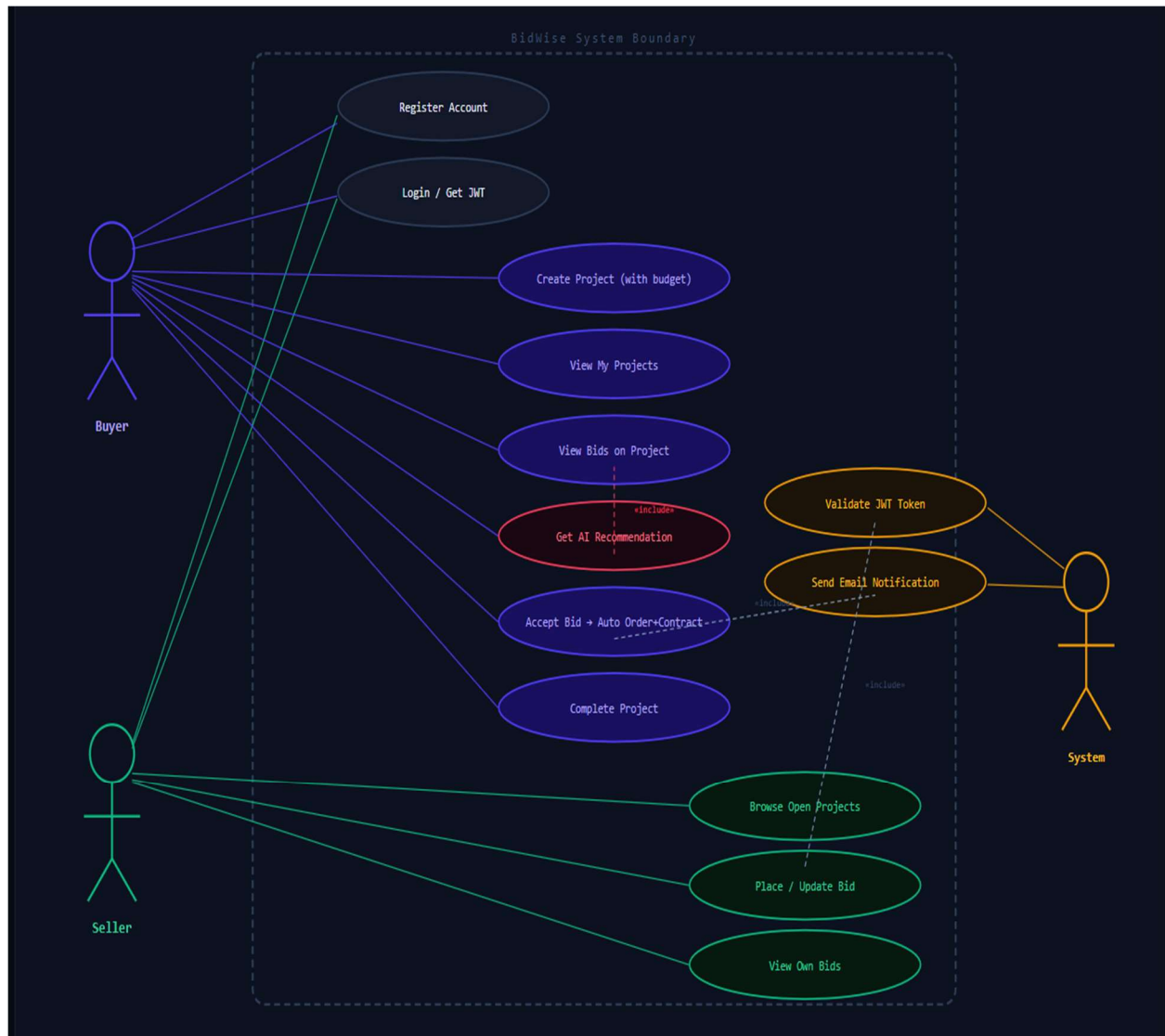
Table 4.1: Core Entity Relationships

Entity	Relationship	Target Entity
User	1 to Many	Project (buyer)
User	1 to Many	Bid (seller)
Project	1 to Many	Bid
Bid	1 to 1	Order (accepted)
Order	1 to 1	Payment
User	1 to Many	Notification
User	1 to Many	Review

4.6 Use Case Design

Use cases define interactions for buyers, sellers, and administrators. Buyers create projects, compare bids, and accept a vendor. Sellers place bids and update order status. Administrators verify sellers, moderate projects, and resolve disputes. This design ensures responsibilities are clear and minimizes cross role leakage.

Figure 4.6: Use Case Diagram



4.7 Class Diagram Design

The class design reflects domain entities and service layers. Entities map to database tables; repositories provide persistence abstraction; services enforce business logic; controllers expose REST APIs. This separation ensures maintainability and supports unit testing. The class diagram highlights key entity attributes and relationships and shows the mapping between service and repository layers.

Figure 4.7: Class Diagram



4.8 Sequence Diagrams

Sequence diagrams illustrate time ordered interactions among frontend, backend, database, and external services.

Figure 4.8: Bid Acceptance Sequence

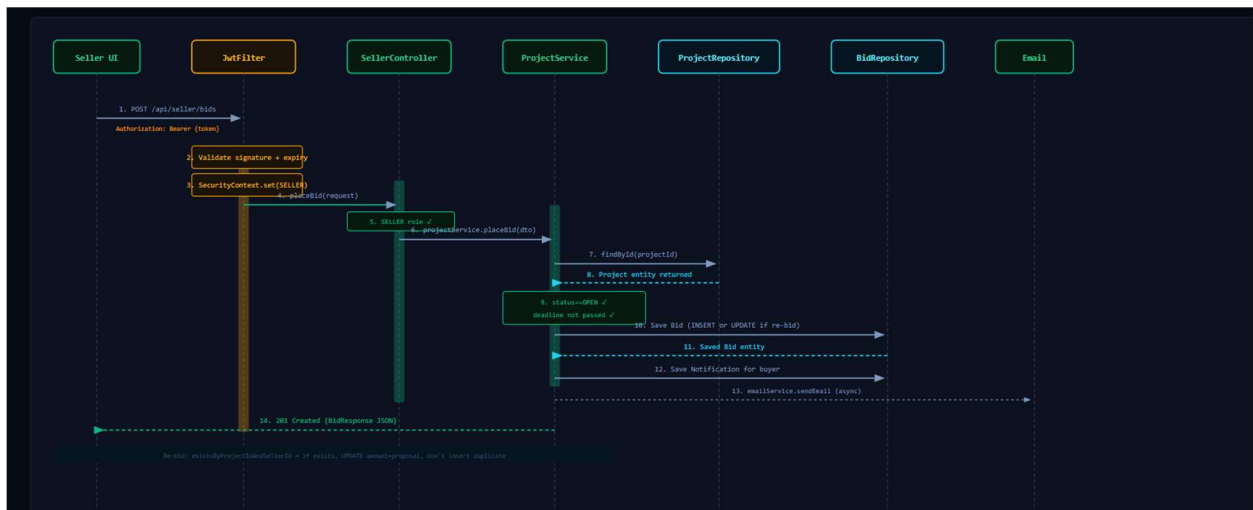
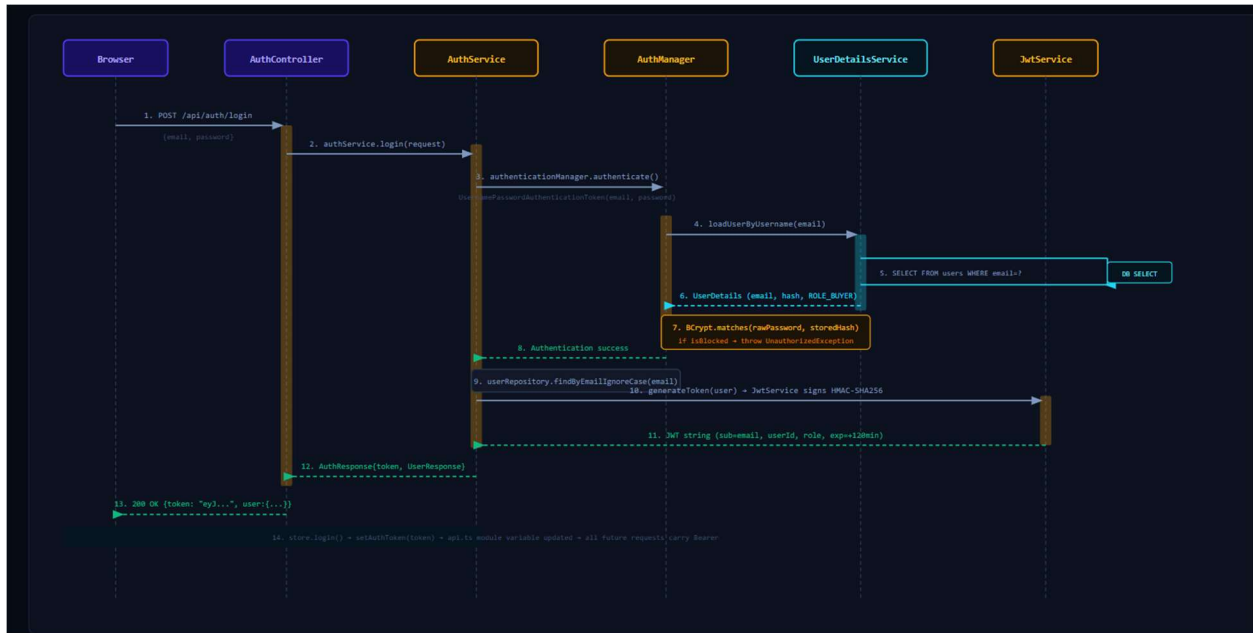


Figure 4.9: Login Sequence



4.9 Deployment Design

Deployment uses a web tier for frontend, an API tier for backend services, and a database tier. The frontend can be deployed as a static build behind a CDN, while the backend runs as a Spring Boot service behind a reverse proxy. Stripe and SMTP are external services accessed securely. The design supports containerization and future cloud deployment on AWS/GCP/Azure.

4.10 Security Design

Security mechanisms include JWT authentication, role based authorization, password hashing, and input validation. Sensitive configuration values (DB credentials, Stripe keys, SMTP passwords) are stored in environment variables rather than code. The payment flow relies on Stripe PaymentIntent and webhook verification to avoid handling raw card data. Bid acceptance and order creation are transactional to prevent partial updates.

Table 4.2: Security Controls

Control	Description
JWT Authentication	Secure session tokens for API requests
Role Based Access	Buyer/Seller/Admin separation
Password Hashing	BCrypt based hashing

Payment Tokenization	Stripe PaymentIntent avoids card storage
Audit Logging	Bid status and order updates stored

4.11 Performance and Scalability

The system is designed for efficient retrieval of project and bid data through pagination and indexed queries. The API is stateless, enabling horizontal scaling. Future improvements can include caching frequently accessed lists and using message queues for notifications to reduce synchronous workload.

4.12 User Interface Design

The UI follows a dashboard style layout with sidebar navigation, cards, and tables. Buyer dashboards show KPI cards and active projects with countdown timers. Sellers see open project listings and bid status. Admin dashboards display metrics and moderation actions. Visual consistency is achieved with Tailwind CSS and ShadCN UI components.

Figure 4.12: Buyer Dashboard Wireframe

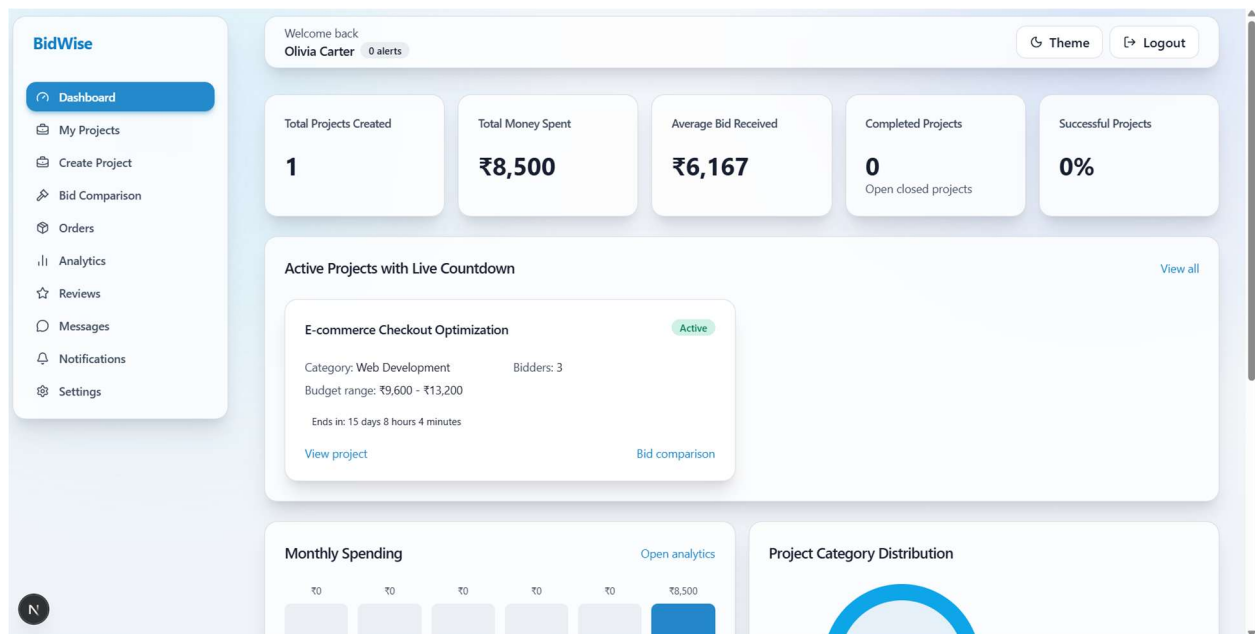
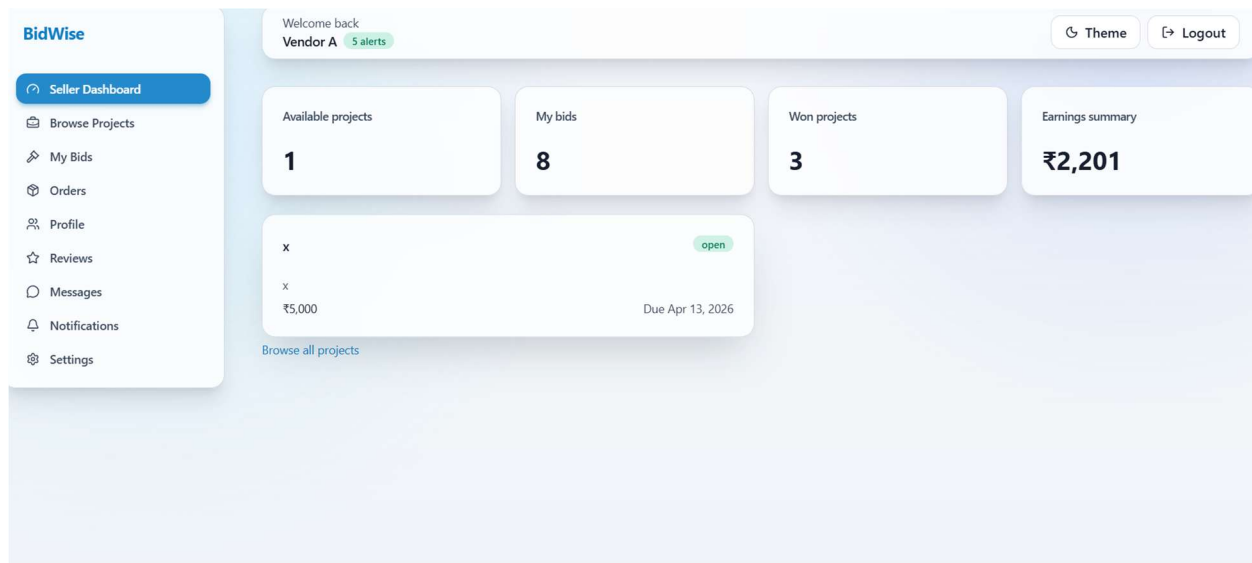


Figure 4.13: Seller Dashboard Wireframe (Placeholder)



4.13 Integration Design

Stripe integration uses PaymentIntent creation at bid acceptance and webhook validation for payment confirmation. Email integration uses SMTP configuration in Spring Mail to send notifications for key events. In app notifications are stored in the database and retrieved via API. The integration points are modular, enabling replacement or extension without core code changes.

4.14 Error Handling and Recovery

The system uses consistent error responses with descriptive messages. Validation errors are returned with HTTP 400 status codes. Unauthorized requests return HTTP 401/403. Critical workflows like bid acceptance and payment updates are wrapped in transactions to avoid partial state changes. Notification failures do not block core transactions but are logged for retries.

4.15 Summary

The BidWise system design combines modular architecture, secure integration, and user centric UI patterns. The design ensures that reverse bidding is not only transparent but also enforceable through structured workflows and lifecycle aware data models. The following chapter details the implementation methodology and development approach.

Chapter 5

Methodology

5.1 Development Approach

The BidWise project followed an Agile inspired development approach with iterative releases and short feedback cycles. The work was organized into feature focused sprints (e.g., authentication, project creation, bidding, order workflow, payment integration, notifications, admin tools). Each sprint produced a working increment that was validated through UI walkthroughs and API testing before new features were layered on top.

Agile was selected because BidWise required frequent refinements in UI/UX, evolving business logic (bid acceptance rules, order status transitions), and integration testing with external services (Stripe and SMTP). A rigid waterfall approach would have delayed critical feedback from the UI and database workflows. Agile allowed early validation of core workflows and reduced risk when extending the system with advanced features such as disputes and analytics.

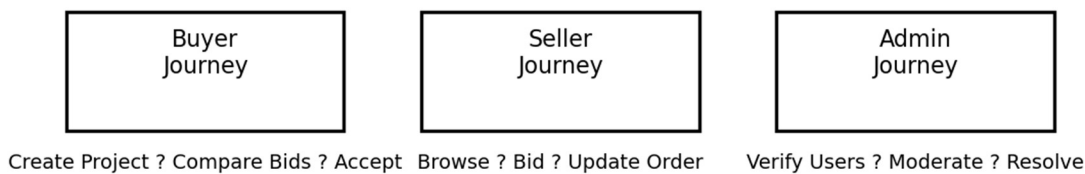


5.2 Design Thinking and User Centric Principles

User centric design principles were applied to prioritize clarity and task completion. Buyer journeys were modeled from project creation to bid acceptance and order completion. Seller

journeys were modeled around discovering projects, placing bids, and tracking deliveries. Admin journeys focused on oversight tasks such as verification and dispute handling. These journeys guided UI layouts, the placement of actions, and the data shown on dashboards.

The design follows a clear SaaS dashboard pattern with a sidebar navigation, KPI cards, and structured tables. Bid comparison views emphasize project specific evaluation, reducing confusion caused by mixing bids across projects. The interface design was refined based on usability testing and feedback on readability, theme compatibility, and workflow clarity.



5.3 Technology Stack Selection

The technology stack was chosen to balance rapid development, scalability, and maintainability. Next.js with TypeScript provides a modern component based UI with type safety and server side rendering support. Tailwind CSS and ShadCN UI enable consistent theming and responsive layouts. Zustand offers predictable state management for complex workflows such as bid acceptance and order updates.

Spring Boot was selected for backend development due to its strong ecosystem for REST APIs, JWT security, and database integration with JPA/Hibernate. MySQL ensures transactional consistency for bids, projects, and orders. Stripe was used for secure payments because it avoids direct handling of card data. Spring Mail integrates transactional email delivery for key events.

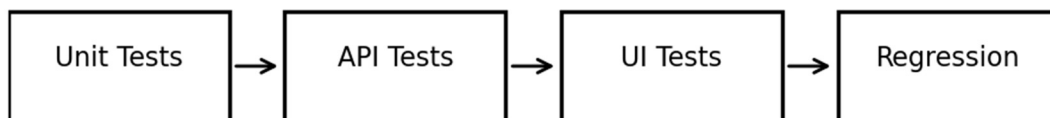
Table 5.1: Technology Stack and Rationale

Layer	Technology	Reason for Selection
Frontend	Next.js + TypeScript	Component?based UI, type safety, routing
Styling	Tailwind CSS + ShadCN	Rapid UI development, consistent design
State	Zustand	Lightweight global state with persistence
Backend	Spring Boot	Robust REST APIs, security support
ORM	JPA/Hibernate	Database abstraction, rapid CRUD
Database	MySQL	ACID transactions, wide adoption
Payments	Stripe	Secure, PCI?compliant payments
Email	Spring Mail	SMTP?based notifications

5.4 Quality Assurance and Testing Strategy

Quality assurance combined manual testing, API validation, and targeted unit tests. Manual testing verified end to end workflows such as bid acceptance and order completion. API endpoints were validated using Postman to ensure correct status transitions and error handling. Unit tests were focused on service layer methods with business logic (e.g., bid acceptance, order status updates, payment intent creation).

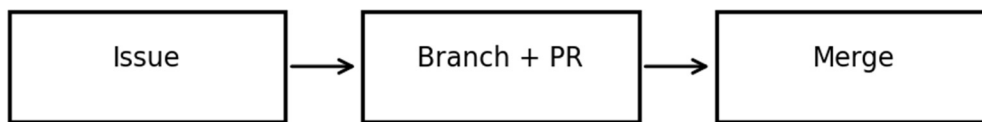
In addition, regression testing was performed after major feature additions to ensure earlier workflows remained intact. Mock data seeding was used during development, then replaced with database backed integration to verify real world behavior. UI consistency testing ensured dark/light themes remained usable and accessible.



5.5 Collaboration Tools and Version Control

GitHub was used for version control, branching, and collaboration. Issues and commits were tracked to maintain accountability for features and bug fixes. For sprint planning, tasks were organized in a backlog and executed in incremental milestones. Documentation was maintained in parallel with implementation to ensure project traceability.

If required, tools such as Trello or Jira can be integrated for advanced sprint planning, but GitHub Issues provided sufficient workflow management for the academic scope.

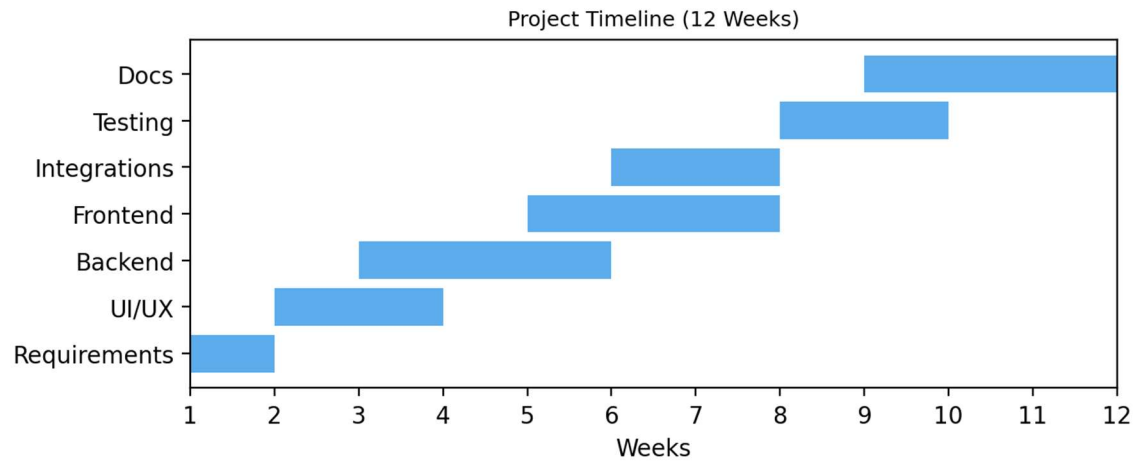


5.6 Project Timeline and Gantt Chart

The project timeline was structured over 12 weeks, aligning with academic semester milestones. Early stages focused on requirements and UI design, followed by backend development, integrations, testing, and documentation.

Table 5.2: Gantt Chart (Project Timeline)

Phase	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12
Requirements & Planning	X											
UI/UX Design		X	X									
Backend Core APIs			X	X	X							
Frontend Dashboards					X	X	X					
Integrations (Stripe/Email)						X	X					
Testing & QA							X	X				
Documentation									X	X	X	X



5.7 Summary

The methodology combined Agile development, user centric design, and structured testing to deliver a robust reverse bidding platform. The iterative process ensured that functional requirements were validated early and refined through repeated UI and API testing. The selected technology stack provided scalability, rapid development, and integration capability suitable for production ready systems.

Chapter 6

6.1 Implementation Overview

BidWise has been implemented as a full-stack reverse bidding marketplace with a modern web frontend and a Spring Boot backend. The implementation follows a layered architecture and clean separation of concerns. Core modules include user authentication, project creation, bid submission, bid comparison, order lifecycle management, Stripe-based payments, in-app notifications, and messaging. The system is fully functional using real database persistence, enabling end-to-end workflows from project posting to payment confirmation and delivery completion.

6.2 Backend Implementation

The backend is implemented using Java Spring Boot with a standard Controller-Service-Repository structure. JPA/Hibernate provides ORM-based persistence to a MySQL database, while Spring Security and JWT tokens handle authentication and authorization. Business rules such as bid acceptance, order creation, and payment recording are enforced in transactional service methods to ensure consistency.

6.2.1 Data Model and Persistence

The database schema represents the marketplace lifecycle. A project belongs to a buyer and can receive multiple bids from sellers. When a bid is accepted, an order is created linking the project, the accepted bid, the buyer, and the seller. Payments and notifications are stored as separate entities to enable audit trails and real-time updates.

Table 6.2 Key Database Tables

Table	Primary Fields	Purpose
users	id, name, email, role, password_hash, created_at	Stores buyer/seller/admin accounts
projects	id, buyer_id, title, description, budget, status, deadline	Buyer project requirements
bids	id, project_id, seller_id, price, status, delivery_days, created_at	Seller bids for projects

Table	Primary Fields	Purpose
orders	id, project_id, bid_id, buyer_id, seller_id, price, status, created_at	Order lifecycle after bid acceptance
payments	id, order_id, buyer_id, seller_id, amount, status, stripe_payment_intent_id	Stripe payment records
notifications	id, user_id, title, message, is_read, created_at	In-app alerts
messages	id, sender_id, receiver_id, body, created_at	Buyer–seller chat messages
reviews	id, project_id, reviewer_id, reviewee_id, rating, comment, created_at	Ratings and reviews between users

6.2.2 REST API Layer

The API layer exposes secure endpoints for project management, bidding, order status updates, payment creation, and notifications. Controllers validate inputs, while services encapsulate domain logic and persist changes within transactions to prevent inconsistent state.

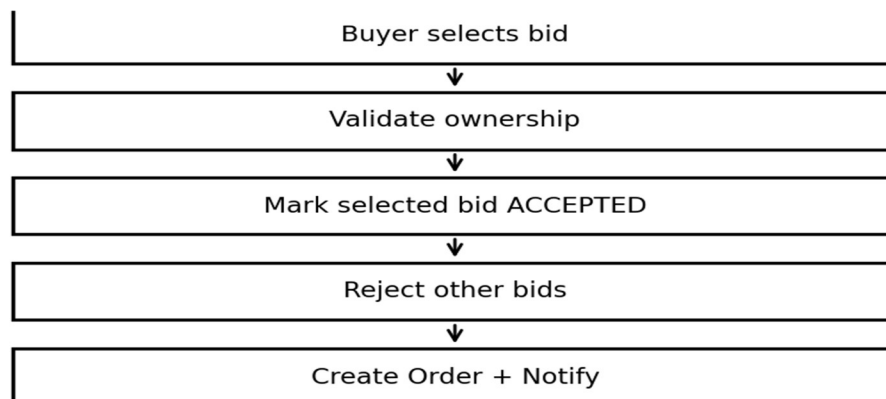
Endpoint	Method	Description
/api/auth/login	POST	User login and JWT issuance
/api/projects	POST/GET	Create project / list projects
/api/projects/{id}	GET	Project details and bids
/api/bids	POST	Place bid on a project
/api/bids/{id}/accept	POST	Accept bid and create order
/api/orders/{id}/status	PUT	Seller updates order status
/api/orders/{id}/complete	POST	Buyer confirms delivery
/api/payments/create-intent	POST	Stripe PaymentIntent creation
Table	Primary Fields	Purpose
users	id, name, email, role, password hash	Stores buyer/seller/admin accounts
projects	id, buyer_id, title, budget, status, deadline	Project requirements posted by buyers
bids	id, project_id, seller_id, price, status, delivery_days	Seller bids for projects

orders	id, project_id, bid_id, buyer_id, seller_id, status	Order lifecycle after acceptance
payments	id, order_id, amount, status, stripe_payment_intent_id	Stripe payment records
notifications	id, user_id, title, message, is_read, created_at	In-app alerts
messages	id, sender_id, receiver_id, body, created_at	Chat messages between users
Module	Backend Implementation	Frontend Implementation
Authentication & Users	JWT auth, user roles, profile endpoints	Login/register UI, role selection, profile settings
Projects	Project entity, CRUD APIs, deadline tracking	Create project form, project list & details
Bids	Bid entity, place/accept/reject logic	Bid cards, comparison table, sorting
Orders	Order entity, status update APIs	Order timeline, delivery confirmation
Payments	Stripe PaymentIntent, webhook, payment table	Payment status badges, checkout UI
Notifications & Chat	Notification entity, message store, email triggers	Notifications list, chat interface

6.2.3 Bid Acceptance and Order Creation

When a buyer accepts a bid, the system performs the following within a single transaction: (1) mark the selected bid as ACCEPTED, (2) reject other bids for the same project, (3) update the project status to IN_PROGRESS, (4) create a new Order entity, and (5) trigger notifications and payment creation. This ensures that the marketplace state remains consistent even under concurrent activity.

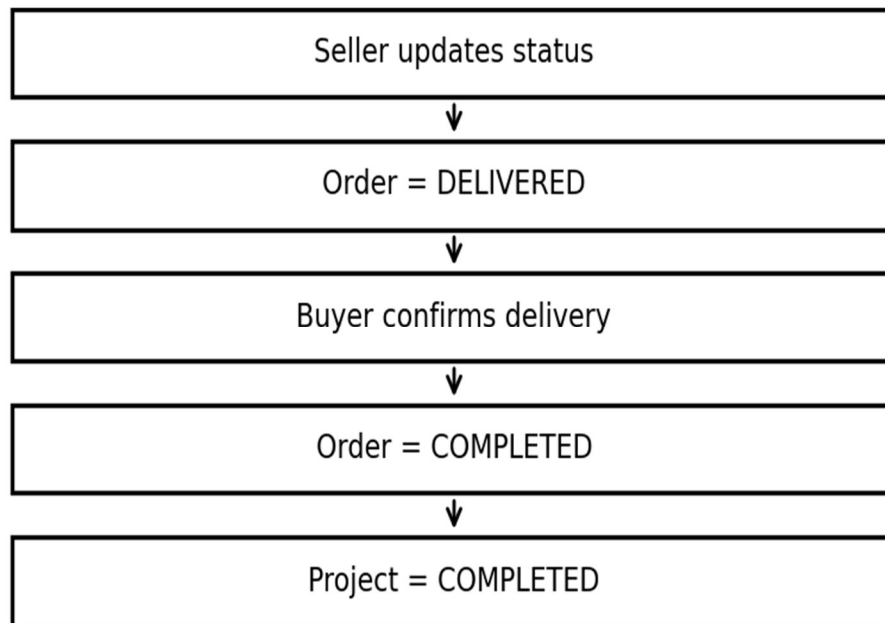
Figure 6.1 Bid Acceptance and Order Creation Flow



6.2.4 Order Lifecycle Management

The order lifecycle progresses through predefined states: CREATED PREPARING SHIPPED DELIVERED COMPLETED. Only sellers can update intermediate states, while buyers confirm final completion. The system logs each update with timestamps and generates notifications for both parties.

Figure 6.2 Order Status Lifecycle Flow



6.2.5 Stripe Payment Integration

Payment is initiated immediately after bid acceptance. The backend generates a Stripe PaymentIntent and stores its identifier in the payments table. Webhook callbacks update payment status to PAID, which triggers a notification to the seller and unlocks order execution. This design ensures payment traceability while keeping sensitive keys on the server side.

6.2.6 Notifications and Email Service

In-app notifications are stored in the database for events such as bid acceptance, order shipment, and payment confirmation. Email notifications are sent using Spring Boot Mail to provide immediate alerts to buyers and sellers. The notification service centralizes message templates to ensure consistent communication.

6.3 Frontend Implementation

The frontend is implemented using Next.js (App Router), TypeScript, Tailwind CSS, and ShadCN UI. Reusable components are organized by feature, and the layout uses a persistent sidebar and top navigation to provide a dashboard-like SaaS experience. State management relies on a centralized store to keep bid, project, and order views synchronized.

6.3.1 Application Structure

Pages are grouped by role (buyer, seller, admin) with shared components for cards, tables, and modal dialogs. Routing follows the App Router conventions, allowing clean separation between dashboards, project details, and workflow-specific pages such as bid comparison and order tracking.

6.3.2 State Management and Data Fetching

A centralized state store manages authentication status, project lists, bids, and order updates. API services are encapsulated into reusable functions that return typed responses. Optimistic UI updates are applied for actions such as bid placement and bid acceptance, while error states are handled with inline alerts and retry options.

6.3.3 UI Components and Theming

The UI uses a card-based layout with responsive grids and consistent spacing. Tables include sorting and filtering for bid comparison. Theme support ensures light and dark modes remain readable and visually consistent. Currency formatting throughout the UI uses Indian Rupees (₹), aligned with the projects target users.

6.3.4 Messaging and Notifications UI

The chat interface provides a real-time style layout with message history, file attachment UI, and status indicators. Notifications are surfaced both in the sidebar and in a dedicated notifications page, with badges for unread items.

6.4 Tools, APIs, and Platforms Used

Category	Tool/Platform	Usage in BidWise
Frontend	Next.js, Tailwind CSS, ShadCN UI	UI, routing, styling and component library
Backend	Spring Boot, JPA/Hibernate	Business logic and persistence
Database	MySQL	Persistent storage for all

		marketplace data
Payments	Stripe	PaymentIntent and webhook processing
Notifications	Spring Boot Mail	Email alerts to buyers/sellers
Version Control	Git + GitHub	Source control and collaboration

6.5 Code Snippets (Representative)

The following simplified snippets illustrate the core implementation patterns. These are representative and summarized for documentation.

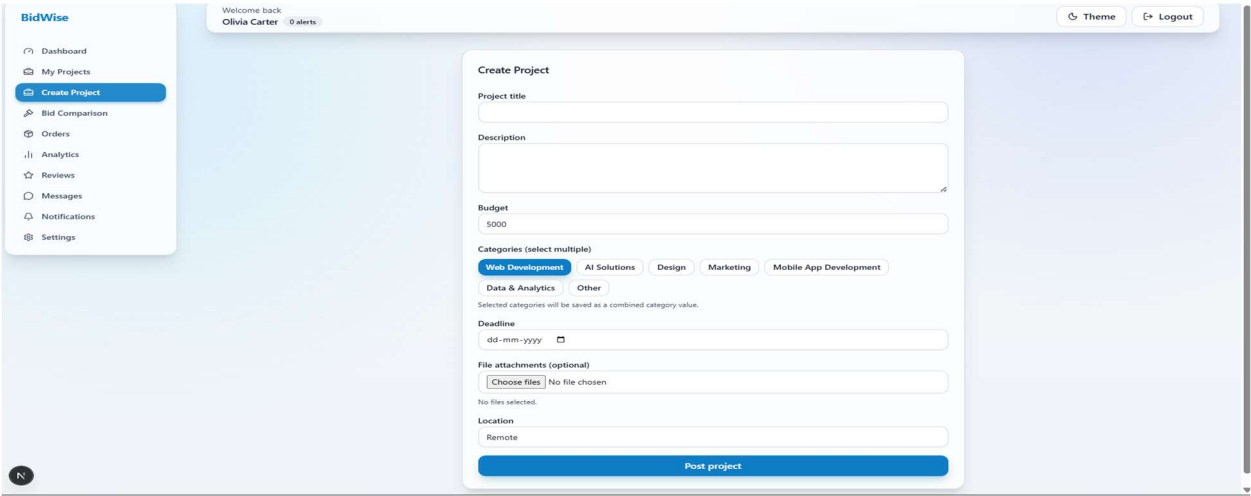
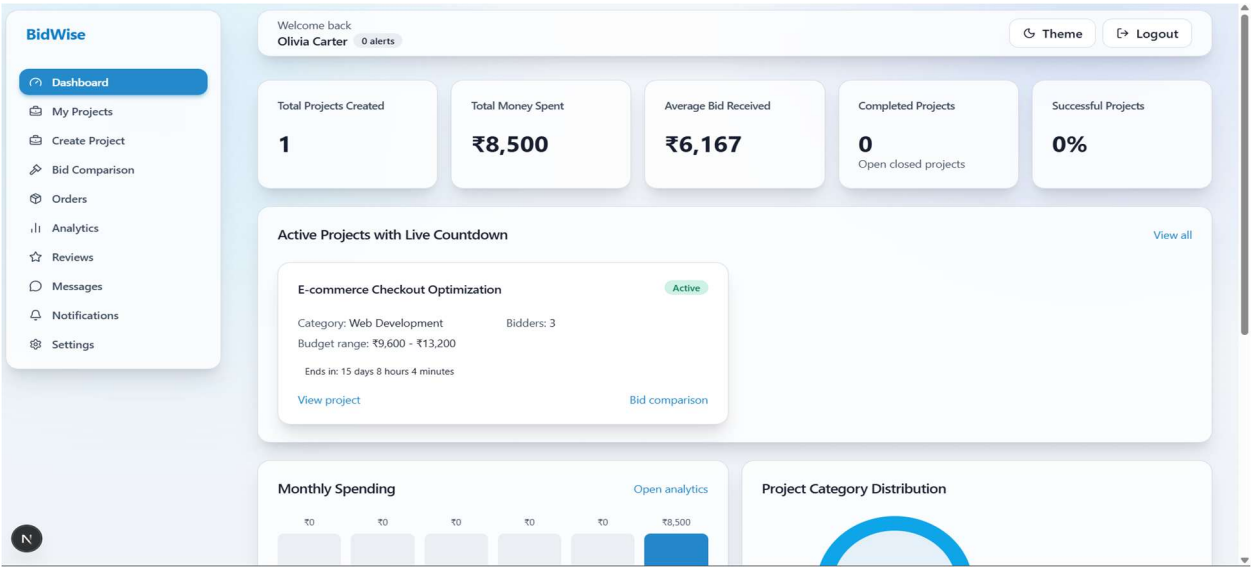
Listing 6.1 Bid Acceptance (Service Layer Pseudocode)

```
public Order acceptBid(bidId, buyerId) {
    Bid bid = bidRepo.findById(bidId);
    Project project = projectRepo.findById(bid.projectId);
    validateBuyer(project, buyerId);
    bidRepo.acceptBid(bidId);
    bidRepo.rejectOtherBids(project.id, bidId);
    project.status = IN_PROGRESS;
    Order order = orderRepo.create(project, bid);
    paymentService.createIntent(order);
    notificationService.notifyBidAccepted(order);
    return order;
}
```

Listing 6.2 Order Status Update (Controller Pseudocode)

```
@PutMapping("/orders/{id}/status")
public Order updateStatus(id, status, sellerId) {
    Order order = orderService.verifySeller(id, sellerId);
    orderService.updateStatus(order, status);
    notificationService.notifyStatusChange(order);
    return order;
}
```

6.6 UI Screenshot Placeholders



BidWise

Dashboard

My Projects

Create Project

Bid Comparison

Orders

Analytics

Reviews

Messages

Notifications

Settings

E-commerce Checkout Optimization - Bid Comparison

Project-specific view

Category: Web Development

Budget: ₹12,000

Bids received: 3

Lowest bid: ₹1,000

[View project details](#) [Back to project list](#)

Bidders

Sort by lowest bid

Sort by highest rating

Sort by fastest delivery

BIDDER NAME	BID AMOUNT	ESTIMATED DELIVERY	SELLER RATING	PAST COMPLETED PROJECTS	LOCATION	RECOMMENDED SCORE	ACTION
Vendor A	Best Value Bid ₹1,000	10 days	4.9/5	0	Remote	90/100	Accept Reject View Profile
Vendor B	₹8,500	19 days	4.6/5	1	India	35/100	Accept Reject View Profile
Vendor A	₹9,000	21 days	4.9/5	0	Remote	30/100	Accept Reject View Profile

Order Details

Completed pending

Created Mar 14, 2026

Project

photo

Buyer: sai

Seller: Vendor A

Price: ₹1

Progress

created

preparing

shipped

delivered

completed

6.7 Flowcharts and Logic Diagrams

Figure 6.3 Marketplace Execution Flow (Project Bid Order Payment)

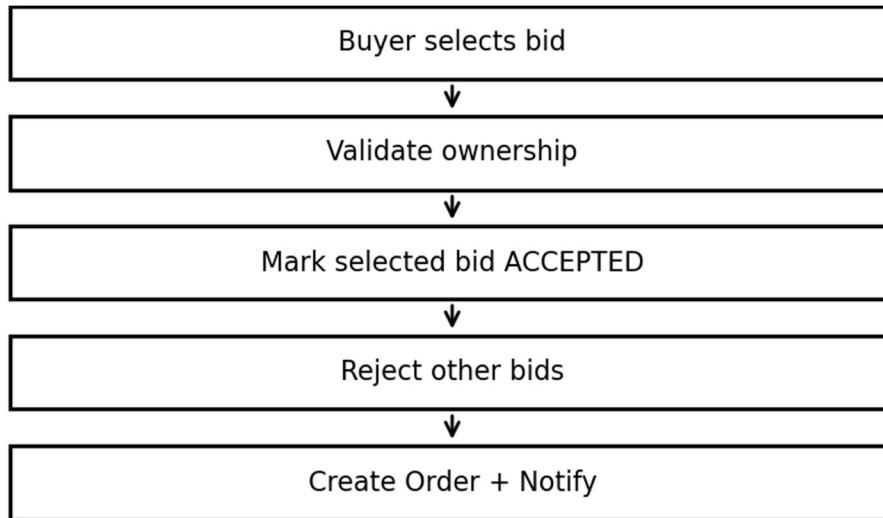
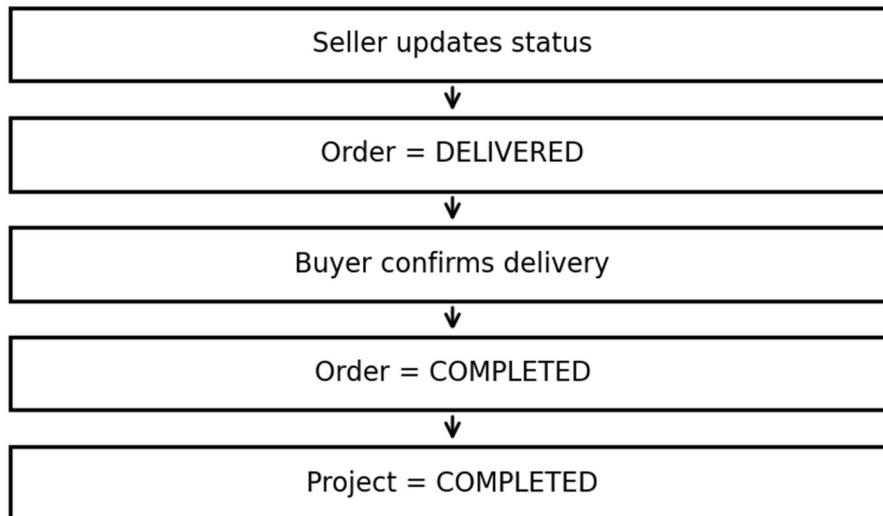


Figure 6.4 Delivery Confirmation Flow

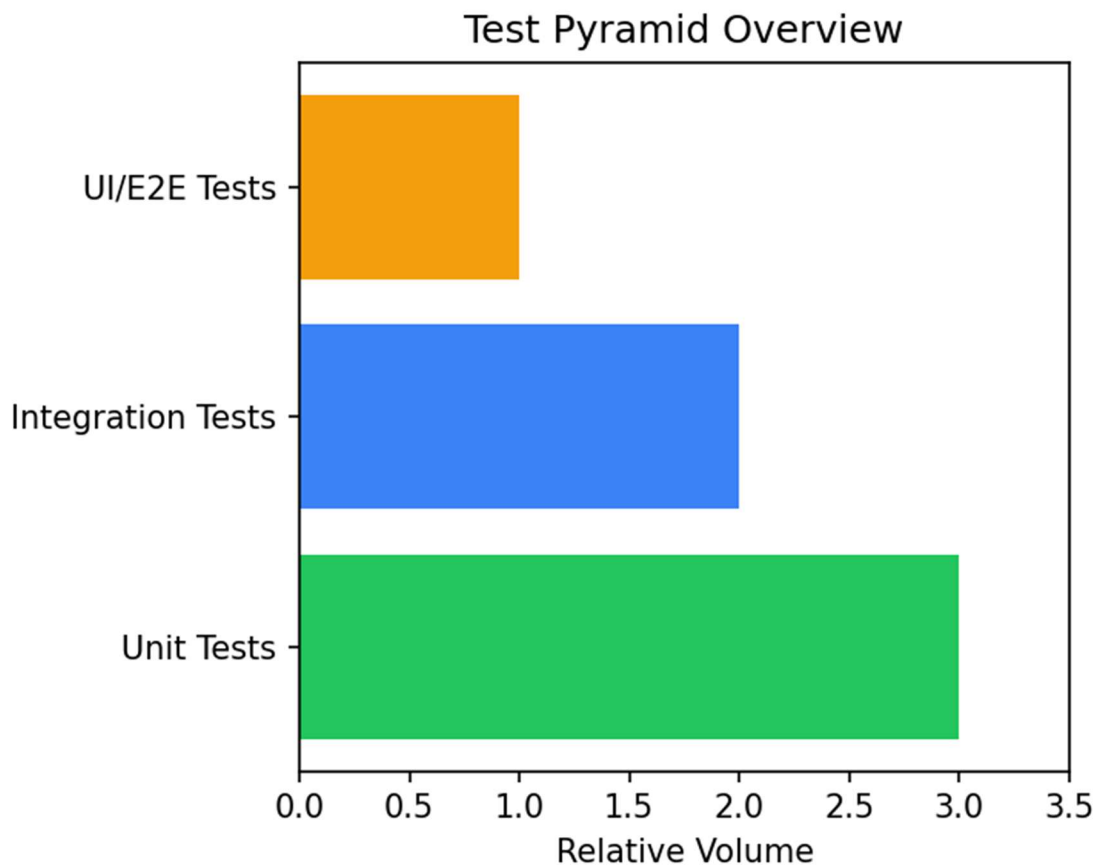


Chapter 7

7.1 Testing Strategy Overview

Testing for BidWise followed a layered strategy to ensure correctness, reliability, and usability. The focus areas included functional workflows (project ? bid ? order), data integrity, access control, and payment confirmation. Testing was executed across unit tests, integration tests, and manual UI validation, with a clear emphasis on critical transactional flows.

Figure 7.1 Test Pyramid for BidWise



7.2 Unit Testing

Backend unit testing was concentrated on service-layer methods that enforce business rules, including bid acceptance, order lifecycle transitions, and payment intent creation. Tests were written using JUnit 5 and Mockito to isolate dependencies. On the frontend, unit tests validated utility functions, state-store updates, and sorting logic in bid comparison tables.

Table 7.1 Unit Test Coverage (Representative)

TC ID	Feature	Input / Action	Expected Result	Status
TC-01	Login	Valid buyer login	JWT issued, dashboard loads	Pass
TC-02	Create Project	Title, budget, deadline	Project stored and listed	Pass
TC-03	Place Bid	Seller submits bid	Bid appears in comparison list	Pass
TC-04	Accept Bid	Buyer accepts bid	Order created, other bids rejected	Pass
TC-05	Payment	Create PaymentIntent	Client secret returned	Pass
TC-06	Order Status	Seller sets SHIPPED	Status updated + notification	Pass
TC-07	Complete Order	Buyer confirms delivery	Order marked COMPLETED	Pass
TC-08	Notifications	Bid accepted	Notification stored and shown	Pass

7.3 Integration and API Testing

Integration tests validated that controllers, services, and repositories interact correctly and persist data as expected. Postman collections and manual API flows were used to verify full workflows such as project creation, bid placement, bid acceptance, and payment intent generation.

Table 7.2 Integration Workflow Validation

Workflow	Steps Verified	Data Check	Status
Project Lifecycle	Create ? View ? Update	Project persisted in DB	Pass
Bid Flow	Place ? View ? Compare	Bid appears in project list	Pass
Bid Acceptance	Accept bid	Order created, other bids rejected	Pass
Payment	Create intent	Payment record stored	Pass
Order Status	Seller updates	Notification + status saved	Pass
Layer	Module	Test Focus	Result
Backend	Bid Service	Accept/reject rules	Pass

Backend	Order Service	Status transitions	Pass
Backend	Payment Service	Stripe intent creation	Pass
Frontend	Utilities	INR currency formatting	Pass
Frontend	Store	Optimistic bid updates	Pass

7.4 Functional Test Cases

Table 7.3 Functional Test Case Summary

TC ID	Feature	Input / Action	Expected Result	Status
TC-01	Login	Valid buyer credentials	JWT issued, dashboard loads	Pass
TC-02	Create Project	Title, budget, deadline	Project stored and listed	Pass
TC-03	Place Bid	Seller submits bid	Bid appears in comparison list	Pass
TC-04	Accept Bid	Buyer accepts bid	Order created, other bids rejected	Pass
TC-05	Payment	Create PaymentIntent	Client secret returned	Pass
TC-06	Order Status	Seller sets SHIPPED	Status updated + notification	Pass
TC-07	Complete Order	Buyer confirms delivery	Order marked COMPLETED	Pass
TC-08	Notifications	Bid accepted	Notification stored and shown	Pass

7.5 Security and Negative Testing

Security-related tests ensured proper role-based access. Buyers cannot access seller-only endpoints and vice versa. Invalid or expired JWT tokens were rejected with 401 responses. Negative test cases such as accepting a bid twice, or updating order status by an unauthorized user, correctly returned validation errors.

7.6 External Validation and Comparison

External validation focused on verifying system outputs against independent references rather than external calculators. Payment confirmation was cross-checked with Stripe dashboard logs, bid ranking was manually verified, and countdown timers were validated against system time.

Table 7.4 External Validation Checks

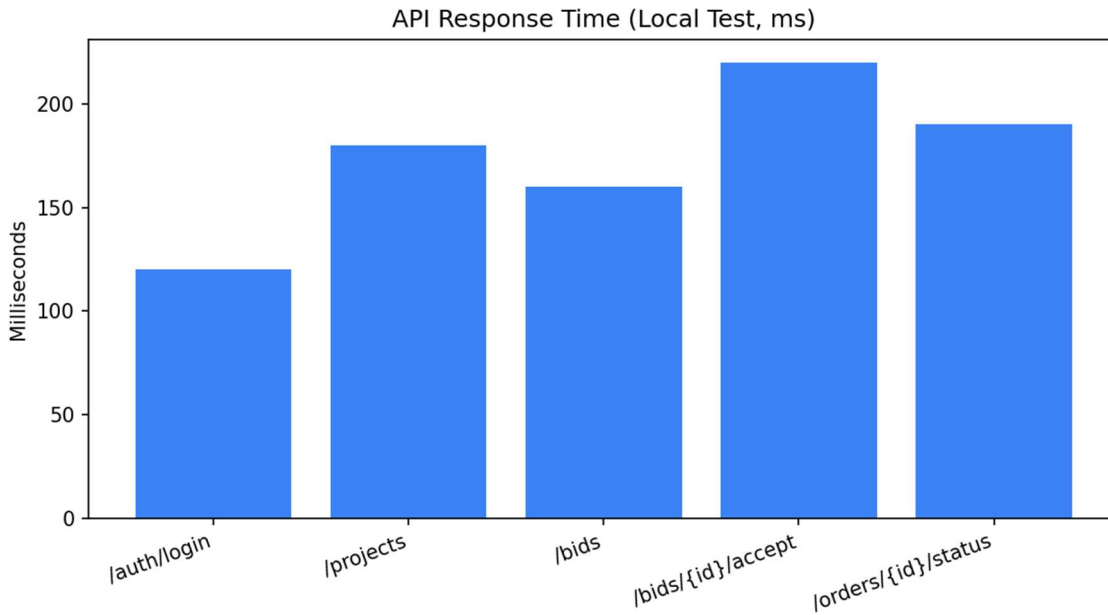
Validation Item	External Reference	Comparison Method	Outcome
Payment status	Stripe Dashboard	Webhook status vs UI badge	Matched
Bid ranking	Manual calculation	Lowest bid + rating sort	Matched
Countdown timer	System time	Remaining time check	Matched
Email alerts	SMTP logs	Sent mail vs UI alert	Matched

7.7 Performance and Load Testing

Performance testing was conducted locally using repeated API requests and moderate concurrency (20-30 simultaneous calls). Average response times for key endpoints were within acceptable limits for a prototype deployment.

Figure 7.2 API Response Time Summary (Local Test)

Endpoint	Avg Response (ms)	Notes
/auth/login	120	JWT issued within threshold
/projects	180	Includes pagination
/bids	160	Bid insert + validation
/bids/{id}/accept	220	Transactional update
/orders/{id}/status	190	Notification triggered



Overall testing confirmed that BidWise workflows are consistent across UI and database state. Critical operations such as bid acceptance are atomic, preventing conflicting updates. Payment status and order lifecycle transitions are traceable and properly audited through notifications.

Chapter 8

8.1 Summary of Achievements

BidWise has been implemented as a complete reverse bidding marketplace that covers the full lifecycle of projects, bids, and post-acceptance execution. Buyers can create projects, receive competitive bids, and select a vendor using data driven comparisons. Sellers can discover projects, submit bids, and manage orders through a structured status pipeline. The system includes secure authentication, role-based access, payment initiation through Stripe, and in app as well as email notifications.

From a technical perspective, BidWise integrates a modern Next.js frontend with a Spring Boot backend and MySQL persistence. Core workflows are transactionally consistent, and data integrity is maintained throughout the project-to-order journey. The platforms dashboards and analytics provide buyers with visibility into spending, bid trends, and project outcomes. Overall, the solution demonstrates a full-stack SaaS implementation that reflects real world marketplace behavior.

8.2 How Objectives Were Fulfilled

The objectives defined during analysis were achieved through direct implementation of each functional requirement. A dynamic bidding workflow allows multiple vendors to compete while ensuring only one accepted bid per project. Buyer and seller dashboards were implemented with role-specific KPIs and navigation. The order lifecycle delivers a traceable workflow from acceptance to completion, supported by notifications and payment status visibility.

Security requirements were addressed using JWT-based authentication, server-side validation, and controlled access to APIs. Performance considerations were addressed through efficient database queries, pagination, and optimized UI rendering. The system therefore fulfills both the business and technical objectives described in the SRS.

Table 8.1 Project Goals vs. Accomplishments

8.3 Key Contributions

The project contributes a working reference model for a reverse bidding platform suitable for service procurement. It demonstrates how competitive pricing and vendor selection can be digitized into a transparent process. The platform also contributes a reusable architecture that can be adapted for other marketplace domains such as logistics, maintenance services, and enterprise procurement.

On the engineering side, the project provides a strong example of full-stack integration using modern tools. The use of transactional workflows for bid acceptance and order creation prevents conflicting states and ensures data consistency. The separation of concerns across controller, service, and repository layers supports maintainability.

8.4 Limitations

Although BidWise is functional and production like, several limitations remain. Automated test coverage is not yet exhaustive, and advanced fraud prevention is not included. Payment handling is limited to basic intent confirmation rather than escrow based milestone releases. Dispute resolution workflows are simplified, and compliance features such as tax handling or invoicing are minimal.

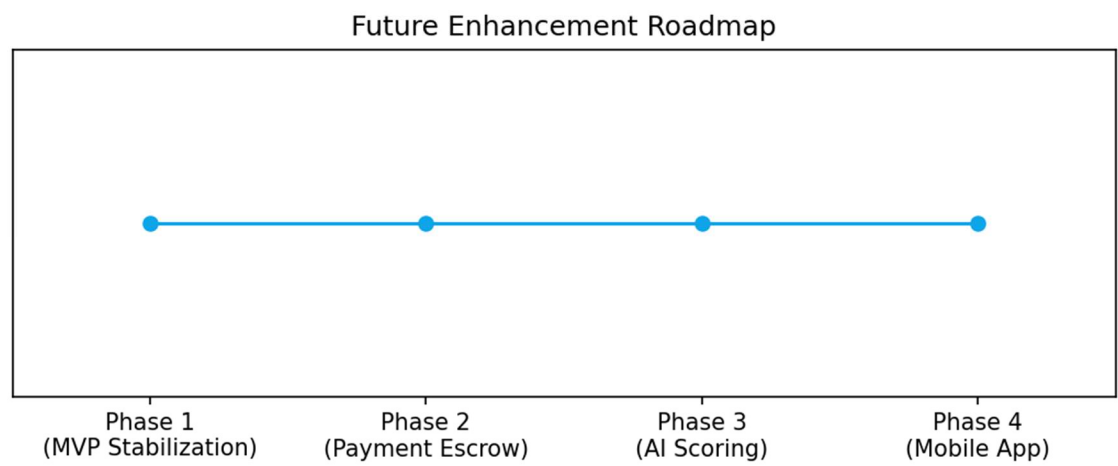
Table 8.2 Limitations and Mitigation Ideas

8.5 Future Enhancements and Additional Modules

Future work focuses on product expansion, advanced trust mechanisms, and scalability. Priority enhancements include escrow or milestone payments, SLA-based delivery enforcement, dispute resolution modules, and AI driven bid scoring that considers pricing, delivery time, and seller reputation.

Additional modules can also include vendor verification workflows, richer analytics dashboards for procurement insights, and advanced notification rules. A dedicated mobile application would improve accessibility and adoption. Integration of file storage services for contracts and attachments would further professionalize the workflow.

Figure 8.1 Future Enhancement Roadmap



8.6 Possible Integrations with Other Tools

To prepare BidWise for real-world enterprise use, integration with external platforms is recommended. Examples include Slack or Microsoft Teams for instant alerts, Jira or Trello for project task synchronization, CRM systems for vendor and buyer management, and cloud storage (S3, Google Drive) for secure document handling.

Payment workflow can be extended with accounting integrations such as QuickBooks or Tally, enabling invoice generation and automated reconciliation. Compliance integrations for GST reporting and audit logs can further support enterprise adoption.

8.7 Closing Remarks

BidWise demonstrates the practical feasibility of reverse bidding in a modern SaaS marketplace. It meets the intended academic and technical objectives while providing a strong foundation for future productization. With further improvements in testing, security, and compliance, the platform can scale to meet enterprise procurement needs.

Limitation	Impact	Mitigation
Limited automated testing	Higher QA cost	Expand unit + integration tests
Basic fraud controls	Potential misuse	Add KYC and anomaly detection
No escrow payments	Risk in high-value work	Introduce milestone escrow
Simplified dispute handling	Limited buyer/seller mediation	Add dispute workflows
Goal	Expected Outcome	Achievement
Reverse bidding workflow	Project ? bids ? selection	Implemented and validated
Role-based dashboards	Buyer/Seller/Admin views	Implemented
Order lifecycle	Post-acceptance tracking	Implemented with status updates
Payments	Stripe-based payment intent + webhook	Implemented
Notifications	In-app + email alerts	Implemented
Analytics	Spending and bid insights	Implemented
Limitation	Impact	Mitigation
Limited automated testing	Higher manual QA effort	Expand unit and integration tests
Basic fraud controls	Potential misuse	Add verification and anomaly detection
No escrow or milestone payments	Risk in large projects	Introduce staged payments
Goal	Planned Outcome	Achievement
Reverse bidding workflow	End-to-end project bid selection	Implemented and validated

Role-based dashboards	Buyer/Seller/Admin views	Implemented
Order lifecycle	Post-acceptance tracking	Implemented with status updates
Payments	Stripe-based intent + webhook	Implemented
Notifications	In-app + email alerts	Implemented
Analytics	Spending and bid insights	Implemented in dashboards

References

The following references are presented in IEEE style and include standards, official documentation, and tools used in the BidWise implementation.

Table R1 Reference Source Summary

[1] Vercel, Next.js Documentation, 2026. [Online]. Available: <https://nextjs.org/docs>. [Accessed: 26-Mar-2026].

[2] Meta, React Documentation, 2026. [Online]. Available: <https://react.dev>. [Accessed: 26-Mar-2026].

[3] Microsoft, TypeScript Handbook, 2026. [Online]. Available: <https://www.typescriptlang.org/docs>. [Accessed: 26-Mar-2026].

[4] Tailwind Labs, Tailwind CSS Documentation, 2026. [Online]. Available: <https://tailwindcss.com/docs>. [Accessed: 26-Mar-2026].

[5] shadcn, shadcn/ui Documentation, 2026. [Online]. Available: <https://ui.shadcn.com>. [Accessed: 26-Mar-2026].

[6] P. Henschel, Zustand Documentation, 2026. [Online]. Available: <https://docs.pmnd.rs/zustand>. [Accessed: 26-Mar-2026].

[7] Spring, Spring Boot Reference Documentation, 2026. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>. [Accessed: 26-Mar-2026].

[8] Spring, Spring Security Reference, 2026. [Online]. Available: <https://docs.spring.io/spring-security/reference/>. [Accessed: 26-Mar-2026].

[9] Hibernate, Hibernate ORM User Guide, 2026. [Online]. Available: <https://hibernate.org/orm/documentation/>. [Accessed: 26-Mar-2026].

[10] Oracle, MySQL 8.0 Reference Manual, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>. [Accessed: 26-Mar-2026].

[11] Stripe, Stripe API Reference, 2026. [Online]. Available: <https://stripe.com/docs/api>. [Accessed: 26-Mar-2026].

[12] E. Jones, J. Bradley, and N. Sakimura, JSON Web Token (JWT), RFC 7519, IETF, 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>. [Accessed: 26-Mar-2026].

[13] JUnit, JUnit 5 User Guide, 2026. [Online]. Available: <https://junit.org/junit5/docs/current/user-guide/>. [Accessed: 26-Mar-2026].

[14] Mockito, Mockito Documentation, 2026. [Online]. Available: <https://site.mockito.org>. [Accessed: 26-Mar-2026].

[15] Postman, Postman Learning Center, 2026. [Online]. Available: <https://learning.postman.com>. [Accessed: 26-Mar-2026].

Category	Examples	Purpose
Frontend Docs	Next.js, React, Tailwind, ShadCN	UI framework and component usage
Backend Docs	Spring Boot, Spring Security, Hibernate	API, security, and ORM guidance
Integrations	Stripe API, JWT RFC	Payments and auth standards
Limitation	Impact	Mitigation
Limited automated testing	Higher QA cost	Expand unit + integration tests
Basic fraud controls	Potential misuse	Add KYC and anomaly detection
No escrow payments	Risk in high-value work	Introduce milestone escrow
Simplified dispute handling	Limited buyer/seller mediation	Add dispute workflows
Goal	Expected Outcome	Achievement
Reverse bidding workflow	Project bids selection	Implemented and validated
Role-based dashboards	Buyer/Seller/Admin views	Implemented
Order lifecycle	Post-acceptance tracking	Implemented with status updates
Payments	Stripe-based payment intent + webhook	Implemented
Notifications	In-app + email alerts	Implemented
Analytics	Spending and bid insights	Implemented
Limitation	Impact	Mitigation
Limited automated testing	Higher manual QA effort	Expand unit and integration tests
Basic fraud controls	Potential misuse	Add verification and anomaly detection

No escrow or milestone payments	Risk in large projects	Introduce staged payments
Goal	Planned Outcome	Achievement
Reverse bidding workflow	End-to-end project bid selection	Implemented and validated
Role-based dashboards	Buyer/Seller/Admin views	Implemented
Order lifecycle	Post-acceptance tracking	Implemented with status updates
Payments	Stripe-based intent + webhook	Implemented
Notifications	In-app + email alerts	Implemented
Analytics	Spending and bid insights	Implemented in dashboards